NTNU – Trondheim
Norwegian University of
Science and Technology

Reinforcement Learning for MagLev Control

Linus Edvardsen

December 2025

SPECIALIZATION PROJECT

Department of Engineering Cybernetics
Norwegian University of Science and Technology

Supervisor: Damiano Varagnolo

Preface

This is a Project Thesis in Reinforcement Learning (RL) for Magnetic Levitation (MagLev) control at NTNU as part of the Cybernetics and Robotics study program. It is a part of the development of the "Take-home MagLev lab" development where this project lays the first building blocks for creating a Machine Learning (ML) component to this lab.

This report assumes a basic level of understanding of control systems and ML.

Trondheim, 2025-12-17

Linus Edvardsen

Acknowledgment

I would like to thank my supervisor, Damiano Varagnolo, for his guidance and feedback throughout this project.

I also thank the members of the MagLev group –Marius Jullum Faanes, Thomas Aleksander Jonsson, and Eivind Bruaset– for their cooperation and discussions. I acknowledge Hans Alvar Engmark for developing the simulator that was used in this thesis.

Finally, I thank Sebastien Nicolas Gros and Shambhuraj Vijaysinh Sawant for their advice regarding the application of RL.

L.E.

Abstract

Magnetic levitation systems represent a classic challenge in control engineering due to their inherently unstable and non-linear dynamics. While traditional model-based strategies like Model Predictive Control (MPC) are effective, they often incur high computational costs and rely on precise system modeling. This thesis investigates the feasibility of applying model-free RL, specifically the Proximal Policy Optimization (PPO) algorithm, to control a 6-Degrees of Freedom (DOF) planar MagLev platform known as "Maggy", with the goal of understanding the limitations of such control strategies, especially against MPC approaches, and provide an overview of how to tune RL for this specific platform, to optimize performance.

The research methodology follows a two-phase approach. First, a simplified 1-DOF levitating ball environment is used to benchmark RL against a nonlinear MPC baseline. Results indicate that RL agents can achieve tracking performance comparable to MPC while offering superior real-time inference speeds, averaging approximately 0.20 ms per control step compared to the significantly higher computational load of optimization-based solvers.

In the second phase, the investigation scales to the full 6-DOF MagLev simulator. Two distinct control architectures are developed and evaluated: a hybrid controller where RL augments a Linear-Quadratic Regulator (LQR), and a pure end-to-end RL controller. The hybrid architecture demonstrates robustness and stability at a standard control frequency of 20 Hz. Conversely, the pure RL approach successfully stabilizes the simulated system from scratch but requires a significantly higher control frequency of 1000 Hz and increased Neural Network (NN) capacity to cope with the faster and more complex system dynamics.

The findings suggest that while pure RL is a viable strategy for magnetic levitation, it imposes stringent timing constraints on embedded hardware, particularly when factored in with state estimation latency. This work lays the foundation for a machine learning component in the "Take-home Maglev Lab," providing a verified training pipeline and a roadmap for future sim-to-real transfer through trajectory randomization and observation stacking.

Contents

Preface i
Acknowledgment ii
Abstract iii

1 Introduction **2**
1.1 Background 2
1.2 Objectives 4
1.3 Sustainability Development Goals 4
1.4 Approach 4
1.5 Contributions 6
1.6 Limitations 6
1.7 Outline 6

2 Collaboration **8**
2.1 Reinforcement Learning 8
2.2 `acados` 9
2.3 Model Predictive Control 9
2.4 Estimator/Observer 9

3 Theory **10**

4 System Description and Modeling **14**
4.1 The 1-DOF Levitating Ball Environment 14
4.2 The 6-DOF Maglev Platform ("Maggy") 15

5 Experimental Design **18**
5.1 Phase 1: 1-DOF Benchmarking 18
5.1.1 Training Protocol 18
5.1.2 Experiment 1: Computational Efficiency Analysis 19
5.1.3 Experiment 2: Reference Tracking Performance 20
5.1.4 Experiment 3: Robustness Testing 21

5.2 Phase 2: 6-DOF Stabilization 21
5.2.1 Scenario A: Hybrid Control (RL + LQR) 21
5.2.2 Scenario B: End-to-End Pure RL 22

6 Results **23**
6.1 How does RL compare against MPC in computational time? 23
6.2 Case Study 1: How is Performance on the 1-DOF Levitating Ball Environment Affected by Model Parameters? 24
6.3 Case Study 2: In Which Circumstances is RL Able to Stabilize the Magnetic Levitation (Maglev) Simulator 47
6.3.1 Architectural Complexity and The Stability-Exploration Trade-off 48
6.3.2 Temporal Resolution and Real-Time Constraints 48
6.3.3 Hyperparameter Sensitivity Analysis 49

7 Conclusions **53**
7.1 Summary and Conclusions 53
7.2 Discussion 54
7.3 Lessons Learned 55
7.4 Discussion of Collaboration 55
7.5 Recommendations for Further Work 55

8 AI Declaration **57**

A Acronyms **58**

Bibliography **60**

Chapter 1

Introduction

1.1 Background

MagLev systems are fundamentally open-loop unstable. An object suspended by an electromagnet is subject to a non-linear force and will, in the absence of active control, either collide with the electromagnet or fall due to gravity. The core challenge, therefore, is to implement a robust, real-time feedback control system that can precisely maintain a desired air gap, stabilizing this inherently unstable equilibrium.

Previously, this problem has been addressed using model-based control strategies, such as Proportional-Integral-Derivative (PID) controllers and MPC [1]. While effective, these methods often require a precise mathematical model of the system's dynamics. Deriving such a model is non-trivial, as it must account for complex non-linearities and, especially the MPC, could be computationally expensive.

This thesis investigates an alternative approach: the application of RL to solve the MagLev stabilization problem. The problem is twofold: first, to determine the feasibility of an RL agent learning an effective control policy directly from interaction with a simulation of the system. Second, this project serves to develop a foundational methodology and framework for this application, as part of a broader effort to learn and apply RL to complex systems.

This research is of interest to control engineers and researchers exploring data-driven and model-free control strategies for non-linear, unstable plants. It also serves a specific, long-term practical goal: to lay the essential groundwork for a future RL-based control component within a MagLev microlab [2], which is currently missing.

The scope of this work is carefully delimited. The primary objective is not to immediately surpass the performance of a meticulously-tuned classical controller. Rather, the focus is on the complete pipeline of applying RL to this problem: investigating suitable algorithms, training strategies, and evaluating the resulting policy's ability to achieve stable levitation. This thesis thus acts as a proof-of-concept and a foundational study, establishing a baseline for future re-

search and development of RL-based control for this application.

The challenge of stabilizing unstable, non-linear dynamical systems has traditionally been the domain of model-based control theory. [3] provide a definitive application of nonlinear MPC to magnetic levitation, establishing a rigorous baseline for tracking accuracy and stability against which new methods must be measured. Bridging the gap between classical modeling and learning-based approaches, [4] demonstrated the practical efficacy of using neural networks for system identification and control on a low-cost MagLev platform. Their work validates that data-driven models can effectively capture the complex non-linearities of magnetic suspension without expensive, high-precision hardware. A major difference between these two and the approach taken here is that the magnet was suspended underneath the controlled electromagnet, which simplifies the control task. Regarding the specific 'Maggy' platform utilized in this project, [5] and [6] established the foundational control architecture. Damsgaard implemented Moving Horizon Estimation (MHE) to address the state estimation challenge, while Mjelde demonstrated the efficacy of Robust Model Predictive Control (RMPC) for stabilization. Their work proves that linearized model-based control is viable for this system, providing a verified baseline against which data-driven approaches can be evaluated.

Concurrently, the field of Deep Learning (DL) has enabled RL to scale to high-dimensional continuous control tasks [7, 8]. Within this domain, [9] introduced PPO, a policy gradient method that balances sample efficiency with ease of tuning, making it a standard choice for continuous control problems. However, the application of RL is not merely about algorithm selection. A large-scale empirical study by Google Brain researchers, [10], highlights that implementation detailssuch as code-level optimizations, network architecture, and reward scalingoften have a more significant impact on agent performance than high-level algorithmic choices. This insight motivates the experimental design of this thesis, which focuses on a rigorous sweep of architectural parameters to ensure a fair comparison between the emerging RL approach and the established MPC standard.

While the principles of classical control for MagLev are well-documented and RL algorithms like PPO are established, the specific integration of model-free RL for this particular instability profile remains an open challenge. The primary gap is the lack of a definitive RL implementation for this specific microlab environment. Consequently, the essential groundwork of establishing a robust simulation pipeline and successfully training an agent to maintain stable levitation on this specific plant remains to be done.

More importantly, a rigorous quantitative comparison between a PPO-based agent and the established MPC baseline is missing from the current literature for this system. It remains to be determined if an RL agent, operating without explicit knowledge of the physics, can match the settling time and steady-state error of the rigorous model-based approach. This thesis aims to fill these gaps by implementing the RL control loop and performing a direct benchmarking

analysis against the MPC standard.

1.2 Objectives

The main objectives of this project are:

1. Quantitatively compare RL with MPC across a set of metrics and environments.
2. Stabilizing a levitating magnet with RL.

1.3 Sustainability Development Goals

This project contributes to United Nations' sustainable development goal 4, quality education, through the work with the microlab, and 9, industry, innovation and infrastructure, by providing more knowledge in the intersection of machine learning and control theory.

1.4 Approach

This section outlines the scientific approach and experimental design adopted to achieve the project objectives. To ensure a rigorous evaluation, the methodology operates in two distinct phases that directly correspond to the main goals: a quantitative comparison of control strategies and the stabilization of a non-linear magnetic levitation plant.

The research utilizes two simulation environments of increasing complexity to systematically test the hypotheses.

The 1-DOF Levitating Ball environment serves as an initial, simplified testbed designed to facilitate the learning of RL fundamentals before transitioning to the complex MagLev system. Rather than modeling specific magnetic interactions, this environment simulates the vertical motion of a generic mass subject to gravitational acceleration and a direct, controllable vertical force. This abstraction removes the complexities of spatial non-linearities and magnetic properties, isolating the core tracking challenge. Despite its simplicity, the environment enforces realistic actuator saturation limits to prevent the application of infinite control effort. This clean benchmark allows for rapid iteration and a direct assessment of the RL agent's ability to track continuous sinusoidal trajectories and manage transient step responses in a controlled setting.

The MagLev Platform constitutes the high-fidelity target environment, modeling a planar magnetic actuator designed for precise positioning. This system consists of a single permanent magnet levitated by a specific arrangement of static permanent magnets and active electromagnets, see Fig. 4.1. Unlike the 1-DOF case, the MagLev platform exhibits inherently unstable and

highly non-linear dynamics arising from the complex interactions of the magnetic fields. Stabilizing this system requires the control agent to manage an unstable equilibrium point where minor deviations can lead to rapid divergence, serving as a rigorous test of the agent's robustness and ability to generalize to physical mechatronic systems.

Two primary controller models are evaluated throughout these environments. A MPC serves as the "Gold Standard" performance benchmark. It utilizes a precise mathematical model of the 1-DOF system to achieve near-optimal tracking error, albeit at a higher computational cost. This baseline is compared against a RL agent utilizing the PPO algorithm. To investigate the impact of model capacity on control performance, the study performs a sweep of network architectures, varying the depth (1, 2, and 3 hidden layers) and width (32, 64, and 128 neurons per layer) used to process the state vector observation and output control actions.

Performance is quantified using two key metrics: tracking accuracy and real-time feasibility. Tracking accuracy is measured by the Root Mean Squared Error (RMSE) between the reference trajectory r_k and actual position y_k:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{k=1}^{N}(r_k - y_k)^2} \tag{1.1}$$

Real-time feasibility is assessed via the mean execution time required to compute a single control action, averaged over 10,000 steps on the target hardware.

The experimental design follows a structured progression. Phase 1 focuses on a comparative analysis where agents are tasked with tracking both sinusoidal and sigmoid step references to evaluate continuous tracking and transient response, respectively. A parameter sweep across network widths and depths is conducted to identify the "Pareto frontier" of efficiency versus accuracy.

Phase 2 shifts to the complex MagLev environment to investigate stabilization under unstable, non-linear dynamics. This phase compares two distinct control scenarios:

- **Scenario A (RL + LQR):** The RL agent operates in conjunction with a classical LQR. In this hybrid setup, the LQR provides a baseline stabilizing controller, while the RL agent learns to compensate for non-linearities or residual errors that the linear model neglects.

- **Scenario B (Pure RL):** The RL agent operates alone, directly outputting control currents to the electromagnets. This tests the agent's ability to learn the complete stabilization policy from scratch.

To determine the conditions under which the RL controller can successfully stabilize the system, the experiments include a rigorous sensitivity analysis of network hyperparameters. Specifically, the impact of the actor's initial standard deviation, mini-batch size, and learning

rate are evaluated. Finally, the robustness of the best-performing policies (using the 2-layer, 256-neuron architecture) is stress-tested by initializing the MagLev system at various spatial offsets from the target equilibrium [9].

1.5 Contributions

The new contributions made by this project is an RL-environment for the MagLev simulator as well as an in-simulation working RL-agent for the MagLev.

1.6 Limitations

The primary limitation of this study is the restriction to a simulation environment. While the simulator provides a high-fidelity representation of the MagLev platform's non-linear dynamics, it cannot fully replicate physical phenomena such as sensor noise, actuator latency, and thermal drift. Consequently, the "sim-to-real" transfer gap remains an unverified variable in the proposed control strategies.

Furthermore, the project encountered technical constraints regarding software interoperability. The "Maggy" simulator is natively implemented in MATLAB, whereas state-of-the-art RL frameworks (such as Stable Baselines3) operate in Python. Future iterations should address this by rewriting the simulator in Python or using the MATLAB Engine for Python. Finally, the RL agent's exploration was constrained to the specific reference trajectories defined in the training curriculum; the agent's ability to generalize to unseen, highly irregular trajectories remains a subject for future robustness testing.

1.7 Outline

The remainder of this thesis is structured as follows:

Chapter 2 describes the collaborative nature of the project, outlining the specific research domains of each group member (RL, `acados`, Estimator/Observer, and MPC) and how these components integrated via the common simulation platform.

Chapter 3 provides the theoretical background necessary for the proposed methods, covering NN, the fundamental RL framework, and the specific details of the PPO algorithm.

Chapter 4 details the system modeling and physical descriptions of the two simulation environments used: the simplified 1-DOF levitating ball benchmark and the high-fidelity 6-DOF "Maggy" MagLev platform.

Chapter 5 presents the experimental design, split into two phases. Phase 1 focuses on benchmarking RL against MPC on the 1-DOF system, while Phase 2 investigates stabilization strategies

(Hybrid and End-to-End) on the 6-DOF system.

Chapter 6 reports the results of the experiments, analyzing computational efficiency, reference tracking performance, and the conditions required for successful stabilization of the nonlinear plant.

Chapter 7 summarizes the project's conclusions, discusses the implications of the findings regarding real-time constraints, and offers a roadmap for future work and sim-to-real transfer.

Finally, **Chapter 8** provides a declaration regarding the use of Artificial Intelligence (AI) tools in the production of this thesis.

Chapter 2

Collaboration

For this autumn we have been four students working on the *Take home maglev lab - Maggy*; Eivind Bruaset, Thomas Aleksander Jonsson, Marius Jullum Faanes and myself, Linus Edvardsen. Each one of us has our own research area while working together:

Eivind Bruaset: The `acados` library and solver implementation.

Linus Edvardsen: Reinforcement Learning (RL).

Marius J. Faanes: Model Predictive Control (MPC) formulation.

Thomas A. Jonsson: The estimator/observer design for the system.

To facilitate this collaboration, we have had a supervision meeting with our supervisor Damiano Varagnolo weekly. We have also had sessions together working on software development and structuring of experiments. To have a common platform for bringing these domains together, the 1-DOF simulation environment was developed by this author and served as the common integration platform for the group's control algorithms. Through the semester we worked together on this simulator, implementing our domains in a way they would work together.

The following subsections provide an overview of each member's contributions.

This section has been written all together with equally shared authorship, and is present in all four reports in the same way. Their reports can be found here.

2.1 Reinforcement Learning

Instead of using a classical controller to control the *Maggy*, the objective of this domain was to determine if a NN could be trained to generate the correct input and stabilize the system using a model-free approach. This work utilized the PPO algorithm to learn control policies directly from interaction with the simulator. The research established a development pipeline starting

with a simplified 1-DOF levitating ball environment to tune hyperparameters, before scaling to the full 6-DOF MagLev system. A key part of the collaboration involved benchmarking these RL agents against the MPC standard, specifically comparing tracking accuracy and real-time computational feasibility across various network architectures.

2.2 `acados`

To address the strict real-time requirements of the *Maggy* platform, this domain investigates the `acados` framework as a high-performance alternative to traditional optimization solvers. The objective is to determine if a solver utilizing tailored **QP!** algorithms and optimized linear algebra routines can achieve the control performance and execution speed necessary for stabilizing inherently unstable dynamics. This work implements and evaluates three different MPC architectures using `acados`: standard MPC, Stochastic MPC, and Tube-based MPC. A central part of the project involves benchmarking these `acados`-based controllers against both model-free RL agents and MPC implementations using `CasADi` to compare tracking precision and computational efficiency.

2.3 Model Predictive Control

While the current simulation framework of the *Maggy* utilizes linearization and LQR control, this domain focuses on a higher-fidelity approach. Previous implementations of MPC on this system, specifically by Mikael Mjelde and Ørnulf Damsgaard in their master's theses, relied on linearizing the system dynamics around an equilibrium point. The primary goal for this project was to move beyond linearization and formulate the MPC as a full Nonlinear Program (NLP). Furthermore, to specifically investigate strategies for improving system robustness under uncertainty, three distinct variants were implemented and compared: a standard MPC, a Tube-based MPC, and a Stochastic MPC.

2.4 Estimator/Observer

Similar to the MPC, the previous iterations of the MHE for the *Maggy* were also implemented as a linear optimizer around a linearization point. In this project the MHE was to be developed fully non-linear, and with an accompanying Extended Kalman Filter (EKF) as an alternative non-linear observer for comparison. Because of the promising framework, the MHE was also to be implemented using `acados`. The overall goal was to implement the observers and find out how they behave under various environmental scenarios.

Chapter 3

Theory

This chapter provides the theoretical background for the methods used in this thesis. We begin with an overview of NN as function approximators, introduce the formal framework of RL, and conclude with a detailed look at the PPO algorithm, which forms the key components of our control agent. This should be sufficient to follow this paper, but for more in depth explanations, refer to the sources for each of the following sub-sections. There is also a lot of work being done in various aspects of this field and other interesting articles I have found during my research, though not directly applied, include: [11], [12] and [13].

The following subsections provide brief "in a nutshell" perspectives. At the conclusion of each subsection, we include supplementary resources, ranging from academic texts to intuitive video explanations that were found to be particularly useful for grasping the core concepts.

Neural Networks as Function Approximators

NN, particularly the Multilayer Perceptron (MLP), are computational models inspired by the structure of the human brain [7]. They are powerful function approximators capable of learning complex, non-linear relationships from data. A NN consists of interconnected nodes, or neurons, organized into layers: an input layer, one or more hidden layers, and an output layer. Each connection has an associated weight, and each neuron applies an activation function (e.g., the Rectified Linear Unit (ReLU) or tanh) to its input. The computation for a single layer's forward pass, which calculates the layer's output $\mathbf{a}^{(l)}$ from the previous layer's output $\mathbf{a}^{(l-1)}$, is described by:

$$\mathbf{a}^{(l)} = f\left(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\right) \tag{3.1}$$

where $\mathbf{W}^{(l)}$ is the weight matrix, $\mathbf{b}^{(l)}$ is the bias vector, and $f(\cdot)$ is the non-linear activation function for layer l. The final output of the network is the result of this computation propagated through all layers.

During training, the network's weights and biases are iteratively adjusted using an algorithm

like backpropagation. This process minimizes a loss function, which quantifies the difference between the network's output and the desired target. In the context of this thesis, NNs are not trained on a static dataset but are used to represent the controller's policy and its value function, allowing the agent to learn and generalize from its experiences.

For more in-depth about neural networks check either the Deep Learning book [7] or 3Blue1Brown on YouTube.

The Reinforcement Learning Framework

RL is a paradigm of machine learning where an agent learns to make optimal decisions by interacting with an environment to maximize a cumulative reward signal [8]. This problem is formally described by a Markov Decision Process (MDP), which is defined by a set of states (S), a set of actions (A), a transition probability function (P), and a reward function (R). The agent's goal is to learn a *policy* (π), which is a mapping from states to actions, that maximizes the expected sum of future rewards.

This framework is directly applicable to control problems. For the MagLev system, the "agent" is the RL controller. The "environment" is the physical (or simulated) MagLev platform. A "state" (s_t) is a vector of sensor readings (e.g., levitator position, velocity, and orientation). An "action" (a_t) is the vector of currents applied to the electromagnets. The "reward" (r_t) is a scalar value designed by the engineer to be high when the agent maintains the desired position (e.g., $r_t = -(y_k - r_k)^2$) and low when it fails. In terms of control problems, this means that higher reward is given for reaching the control objective.

The reinforcement learning playlist by Steve Brunton, on Youtube covers this and some of the following sections, while the Reinforcement Learning book [8] is more detailed and also covers this and the agent-environment loop and actor-critic and policy gradient methods.

The Agent-Environment Loop

The learning process in RL follows a distinct "agent-environment loop." At each discrete time step t, the agent observes the current state s_t from the environment. Based on this state, the agent's policy $\pi(a_t|s_t)$ selects an action a_t. This action is executed in the environment, which then computes the system's new dynamics and transitions to a new state, s_{t+1}. Simultaneously, the environment provides the agent with a reward r_{t+1} that evaluates the consequence of the last action. The agent uses this new information (the tuple $s_t, a_t, r_{t+1}, s_{t+1}$) to update its policy, and the loop repeats. This thesis utilizes this exact loop, with the 1-DOF ball and MagLev simulators serving as the "environment."

Actor-Critic and Policy Gradient Methods

RL algorithms can be broadly categorized. A key family is Policy Gradient (PG) methods, which directly optimize the parameters of the agent's policy network to maximize the expected reward. A more advanced and stable variant is the Actor-Critic (AC) architecture [14]. AC methods split the agent into two separate networks:

- **The Actor:** This is the policy network ($\pi_\theta(a|s)$) with parameters θ. It controls the agent's behavior, deciding which action to take in a given state.
- **The Critic:** This is a value network ($V_\phi(s)$) with parameters ϕ. It does not take actions, but instead estimates the *value function* (the expected future reward) from the current state. Its job is to "critique" the Actor's actions by providing a low-variance signal, known as the *advantage* $A(s,a) = Q(s,a) - V(s)$, which measures whether an action was better or worse than expected.

The Actor updates its policy in the direction suggested by the Critic, while the Critic updates its value estimate based on the rewards received from the environment.

Proximal Policy Optimization (PPO)

PPO [9] is a first-order optimization method designed to improve training stability by preventing large updates that could potentially destroy the policy. Unlike standard policy gradient methods, PPO strictly limits how much the policy can change in a single step using a "trust region" approach implemented via a clipped objective function.

Central to PPO is the probability ratio $r_t(\theta)$, which measures the deviation of the current policy π_θ from the old policy $\pi_{\theta_{old}}$:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \tag{3.2}$$

To avoid excessively large updates when $r_t(\theta)$ deviates significantly from 1, PPO maximizes a *Clipped Surrogate Objective* L^{CLIP}:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{3.3}$$

where $\hat{A}_t$ is the estimated advantage and ϵ is a hyperparameter (typically 0.2) that defines the clipping interval. This objective acts as a pessimistic lower bound, ignoring changes in probability that would improve the objective if they violate the clipping constraint.

For AC architectures, the final optimization objective includes the value function error L_t^{VF}

and an entropy bonus S to ensure sufficient exploration:

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t \left[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t) \right] \tag{3.4}$$

Here, c_1 and c_2 are coefficients weighting the value loss and entropy bonus, respectively.

This is best understood through the cited OpenAI paper [9] or this article.

Chapter 4

System Description and Modeling

This chapter details the physical characteristics and mathematical models of the two dynamic environments utilized in this project: the simplified 1-DOF levitating ball and the 6-DOF "Maggy" platform. The governing equations and physical constraints derived here establish the simulation environments used for the subsequent control verification.

4.1 The 1-DOF Levitating Ball Environment

The 1-DOF environment serves as a fundamental benchmark for verifying RL feasibility against classical control methods. It simulates a point mass m constrained to move along the vertical z-axis, subject to gravitational acceleration g and a controllable vertical force u.

System Dynamics The equation of motion is governed by Newton's second law:

$$m\ddot{z} = u - mg \tag{4.1}$$

where $u \in [u_{min}, u_{max}]$ represents the control input (force) limited by actuator saturation. The system is subject to a hard physical constraint at $z = 0$ (the ground), where the velocity is reset to zero upon collision, simulating an inelastic impact.

RL Formulation To control this system, the problem is formulated as a MDP:

- **Observation Space:** A vector $s_t = [z, \dot{z}, e_z]$, where z is the current height, $\dot{z}$ is the vertical velocity, and $e_z = z_{target} - z$ is the error relative to the reference.
- **Action Space:** A continuous scalar $a_t \in [-1, 1]$, which is mapped linearly to the force limits of the actuator u.

- **Objective:** The agent must track varying reference trajectories while avoiding ground collisions and leaving at the top of the screen.

The specific physical parameters used for the 1-DOF simulation are detailed in Table 4.1.

Table 4.1: Physical Parameters of the 1-DOF Environment

Parameter	**Value**
Nominal Mass	1.0 kg
Gravitational Acceleration	9.81 m/s^2
Force Saturation Limits	±50 N
Ground Height	0.0 m
Max Height	600.0 m
Simulation Frequency	20 Hz

The implementation of this system in Python code can be found on GitHub

4.2 The 6-DOF Maglev Platform ("Maggy")

The target environment is a high-fidelity simulation of the "Maggy" platform [1], illustrated in Fig. 4.1. Unlike the 1-DOF case, this system represents a Multiple-Input Multiple-Output (MIMO) control problem characterized by open-loop instability and strong nonlinearities.

Physical Configuration The platform consists of a floating permanent magnet (the "mover") suspended above a base containing a specific array of static permanent magnets and four active air-cored solenoids. The static magnets provide a baseline repulsive force that counteracts gravity but introduces an unstable equilibrium. The four solenoids provide the active control authority necessary to stabilize the mover in 6 DOF.

System Dynamics The state of the system is defined by a 12-dimensional vector:

$$\mathbf{x} = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\psi}]^T \tag{4.2}$$

representing the position, Euler angles (orientation), and their respective time derivatives. The control input is a 4-dimensional vector $\mathbf{u} = [i_1, i_2, i_3, i_4]^T$, representing the currents applied to each solenoid. The magnetic forces and torques acting on the mover are highly nonlinear functions of the air gap and the coil currents, making linear control techniques valid only within a small region around the equilibrium point.

The rigid body dynamics are governed by the Newton-Euler equations of motion. Defined about the center of mass, the dynamics are governed by:

$$\begin{bmatrix} \mathbf{F}(\mathbf{x},\mathbf{u}) - \mathbf{F}_g \\ \boldsymbol{\tau}(\mathbf{x},\mathbf{u}) \end{bmatrix} = \begin{bmatrix} m\mathbf{I}_3 & \mathbf{0}_{3\times3} \\ \mathbf{0}_{3\times3} & \mathscr{I} \end{bmatrix} \begin{bmatrix} \mathbf{a} \\ \dot{\boldsymbol{\omega}} \end{bmatrix} + \begin{bmatrix} \mathbf{0}_{3\times1} \\ \boldsymbol{\omega} \times \mathscr{I}\boldsymbol{\omega} \end{bmatrix} \tag{4.3}$$

where m is the mass of the magnet, $\mathbf{I}_3$ is the 3×3 identity matrix, and $\mathbf{F}_g = -mg\hat{e}_z$ represents the gravitational force. The vectors $\mathbf{a}$ and $\dot{\boldsymbol{\omega}}$ denote linear and angular accelerations, respectively. The inertia matrix $\mathscr{I}$ for the cylindrical levitating magnet is defined as $\mathscr{I} = \mathrm{diag}([\frac{1}{4}mR^2, \frac{1}{4}mR^2, \frac{1}{2}mR^2])$, where R is the radius of the magnet. The complete system can be represented in a compact state-space form $\dot{\mathbf{x}} = A\mathbf{x} + Bf(\mathbf{x},\mathbf{u})$, where the system matrices A and B are:

$$A = \begin{bmatrix} \mathbf{0}_{6\times6} & \mathbf{I}_6 \\ \mathbf{0}_{6\times6} & \mathbf{0}_{6\times6} \end{bmatrix}, \quad B = \begin{bmatrix} \mathbf{0}_{6\times6} \\ \mathbf{I}_6 \end{bmatrix} \tag{4.4}$$

The nonlinear term $f(\mathbf{x},\mathbf{u})$ encapsulates the inverse dynamics required to solve for the accelerations:

$$f(\mathbf{x},\mathbf{u}) = \begin{bmatrix} m\mathbf{I}_3 & \mathbf{0}_{3\times3} \\ \mathbf{0}_{3\times3} & \mathscr{I} \end{bmatrix}^{-1} \left(\begin{bmatrix} \mathbf{F}(\mathbf{x},\mathbf{u}) \\ \boldsymbol{\tau}(\mathbf{x},\mathbf{u}) \end{bmatrix} - \begin{bmatrix} \mathbf{F}_g \\ \boldsymbol{\omega} \times \mathscr{I}\boldsymbol{\omega} \end{bmatrix} \right) \tag{4.5}$$

The magnetic force $\mathbf{F}$ and torque $\boldsymbol{\tau}$ are calculated by integrating the interaction between the base magnetic field $\mathbf{B}_{\text{base}}$ and the magnetization surface current $\mathbf{K}$ and polarization $\mathbf{J}$ over the surface S of the levitating magnet:

$$\mathbf{F}(\mathbf{x},\mathbf{u}) = \int_S \mathbf{K} \times \mathbf{B}_{\text{base}}\, dS, \quad \boldsymbol{\tau}(\mathbf{x},\mathbf{u}) = \int_S \frac{1}{\mu_0} \mathbf{J} \times \mathbf{B}_{\text{base}}\, dS \tag{4.6}$$

A more rigorous derivation of the dynamics is provided in [1], with the corresponding software implementation detailed in [2].

RL Formulation

- **Observation Space:** The full 12-dimensional state vector $\mathbf{x}$, normalized to assist NN convergence. In the "End-to-End" experiments, this is fed directly to the policy network.
- **Action Space:** A continuous vector $a_t \in [-5, 5]^4$.
- **Control Challenge:** The agent must learn a policy $\pi_\theta(a_t|s_t)$ that simultaneously stabilizes the unstable magnetic equilibrium and tracks position references, a task requiring significantly higher control frequencies (approx. 1000 Hz) compared to the 1-DOF system.

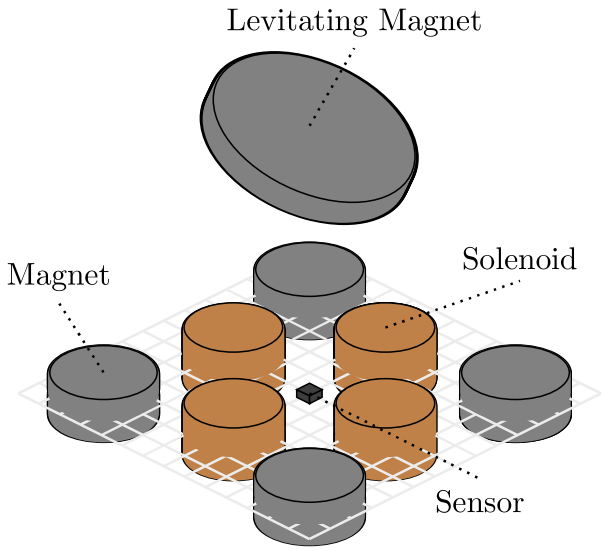


Figure 4.1: Illustration of the *Maggy*. Taken from [2]

Chapter 5

Experimental Design

This chapter details the specific test scenarios and evaluation protocols designed to validate the control strategies defined in Chapter 4. The experiments are divided into two phases: a comparative benchmark on the 1-DOF system and a stabilization study on the 6-DOF MagLev platform.

5.1 Phase 1: 1-DOF Benchmarking

The primary objective of this phase is to quantitatively compare the RL approach against a non-linear MPC baseline.

5.1.1 Training Protocol

To ensure a fair comparison across architectures, all RL agents in Phase 1 follow a standardized training curriculum. A total of 9 distinct agents are trained to evaluate the impact of model capacity:

- **Architectural Variants:** The policy networks are swept across a grid defined by:
  - **Depth (Hidden Layers):** $\{1, 2, 3\}$ layers.
  - **Width (Neurons per Layer):** $\{32, 64, 128\}$ neurons.

  These values were chosen based on the values chosen in [10].
- **Algorithm:** PPO.
- **Training Budget:** 20 million time steps per agent, as this seemed to let all the models have sufficient time in training, so that limited training should not interfere with results.

- **Training Reference:** To ensure robustness and generalization, the agent is trained on a randomized curriculum. At the start of each episode, the environment randomly selects between a *Sine* or *Sigmoid* trajectory, described by:

$$r_{sine}(t) = A\sin(\omega t) + C \tag{5.1}$$

$$r_{sigmoid}(t) = C + A\tanh\left(k\sin(\omega(t - t_0))\right) \tag{5.2}$$

  The parameters are sampled uniformly:

  - **Reference Type:** $Type \in \{\text{Sine}, \text{Sigmoid}\}$
  - **Base Height (Offset):** $C \sim \mathcal{U}(100, 500)$ m
  - **Amplitude:** $A \sim \mathcal{U}(10, 60)$ m
  - **Sine Period:** $T \sim \mathcal{U}(0.5, 5.0)$ s
  - **Sigmoid Slope:** $k \sim \mathcal{U}(5, 15)$
  - **Sigmoid Period:** $T \sim \mathcal{U}(0.2, 1.0)$ s

- **Reward Function:** A dense reward signal is used to guide the learning process, penalizing deviation from the target and catastrophic failures (ground collisions). The exponential keeps the value between 0 and 1.5. This reward function was chosen as it encapsulates what is important in the control scenario:

$$R_t = \begin{cases} -200 & \text{if } z_t \leq 0 \text{ (crash)} \\ 1.5 \cdot e^{-0.05|z_t - r_t|} & \text{otherwise} \end{cases} \tag{5.3}$$

5.1.2 Experiment 1: Computational Efficiency Analysis

This experiment quantifies the real-time feasibility of the RL controllers compared to the MPC baseline.

Objective To determine the mean inference latency for each network architecture and identify which configurations satisfy the hard real-time constraints of the embedded loop.

Methodology The computational cost is evaluated on a standardized testbench (AMD Ryzen 9 5950X, 128GB RAM, NVIDIA GeForce 710 (2GB)). The measurement protocol is as follows:

1. **Sampling:** 1000 randomly generated valid state vectors are prepared.

2. **Timing:** For each architecture, the agent's `predict()` function is executed for each state. The wall-clock time is measured using a high-resolution performance counter.

3. **Aggregation:** The average time per control step is computed.

4. **Baseline Comparison:** These results are compared against some selected MPC solver times from the reports of Bruaset and Faanes.

The code for this is shown in Listing 5.1. One drawback of this is that any overhead from the loop also gets timed. This should be the same for every controller, so while the actual control computation might be lower than the result from this code, the trends that we find still reflects what controllers are the fastest.

```
start_time = time.perf_counter()
for obs in states_to_test:
    _ = agent.predict(obs, deterministic=True)
end_time = time.perf_counter()
total_time = end_time - start_time
avg_time_ms = (total_time / 1000) * 1000
```

Listing 5.1: Timing of control action. The Python time package is used here.

5.1.3 Experiment 2: Reference Tracking Performance

To evaluate both continuous tracking capabilities and transient response, the agents are subjected to two distinct reference signals:

Sinusoidal Tracking Used to test the agent's ability to minimize steady-state error during continuous motion. The reference trajectory is given by:

$$r(t) = A\sin(\omega t) + C \tag{5.4}$$

where A is the amplitude, ω is the frequency, and C is the vertical offset. In the tests $A = 50m$ and $C = 300m$ will be used. $\omega = [0.1, 1, 10]$ are tested to determine the bandwidth limits of the controller. The theoretical maximum upwards acceleration of the ball is given by $F = ma$, $a = 40.19ms^{-2}$, which is equal to an $\omega = 0.8$. The first frequency $\omega = 0.1$ is therefore very attainable to follow, for $\omega = 1$ it should almost be able to follow, while for $\omega = 10$ it will be impossible, but it serves as a stress test for the controller.

Sigmoid Step Response Used to evaluate transient performance, specifically overshoot and settling time. The smooth step reference is defined as:

$$r(t) = \frac{C - A}{1 + e^{-k\sin(\omega_{\text{sig}}(t-\phi))}} + 2A \tag{5.5}$$

where A is the step magnitude, k controls the steepness of the slope, C is the base height offset, ω changes the frequency of the steps and ϕ is the phase shift. In the experiments $A = 50m$, $k = [5, 10, 15]$, $C = 100m$, $\omega = 0.3$ and $\phi = \frac{\pi}{2*0.3}$ will be used. All of these require an upwards force that supersedes the system's capabilities, with $k = 5$ requiring $a = 55.5ms^{-2}$

Disturbance To make the scenarios more realistic and challenging a perlin noise (pnoise1 from the `noise` Python package) was added to the force from the controller. The perlin noise had the following parameters: 4 octaves, frequency 1 and amplitude 4.

5.1.4 Experiment 3: Robustness Testing

To simulate model mismatch, the trained agents are tested in environments where the physical mass of the ball differs from the training mass (1.0 kg). The specific test cases are:

- **Light Mass:** 0.3 kg (High acceleration potential).
- **Heavy Mass:** 1.7 kg (Sluggish response).

5.2 Phase 2: 6-DOF Stabilization

The second phase focuses on stabilizing the inherently unstable 6-DOF MagLev platform. Two distinct control scenarios are investigated to determine the feasibility of RL in this domain. This phase involves extensive hyperparameter optimization and computationally intensive simulation and data-hungry RL.

5.2.1 Scenario A: Hybrid Control (RL + LQR)

In this scenario, the RL agent acts as a supervisory or augmenting controller on top of a baseline LQR.

- **Role:** The LQR provides nominal stability, while the RL agent optimizes for secondary objectives (e.g., minimizing power or correcting non-linear steady-state errors).
- **Objective:** To validate the RL agent's capacity for secondary objective optimization within a stabilized regime, thereby reducing the complexity of the subsequent transfer to pure RL.

5.2.2 Scenario B: End-to-End Pure RL

In this scenario, the RL agent replaces the classical controller entirely, mapping state observations directly to solenoid currents.

- **Role:** Complete stabilization of the unstable equilibrium from scratch.

Chapter 6

Results

6.1 How does RL compare against MPC in computational time?

A key performance metric for real-time control is the computational time required to generate a control action. This section compares the average execution time of the baseline MPC against several trained RL controllers with varying network architectures.

The execution time was measured on the test hardware as described in Experiment 5.1.2. The results are presented in Table 6.1.

Controller Type	**Architecture (Layers x Width)**	**Avg. Time (ms)**
Standard MPC	N=20	10.1335
Standard `acados` MPC	N=20	1.2146
RL (PPO)	1 Layer x 32 Neurons	0.1866
RL (PPO)	1 Layer x 64 Neurons	0.1856
RL (PPO)	1 Layer x 128 Neurons	0.1829
RL (PPO)	2 Layers x 32 Neurons	0.2088
RL (PPO)	2 Layers x 64 Neurons	0.2020
RL (PPO)	2 Layers x 128 Neurons	0.2128
RL (PPO)	3 Layers x 32 Neurons	0.2157
RL (PPO)	3 Layers x 64 Neurons	0.2198
RL (PPO)	3 Layers x 128 Neurons	0.2105

Table 6.1: Comparison of Average Computational Time per Control Action. The used hardware was: AMD Ryzen 9 5950X, 128GB RAM, NVIDIA GeForce 710 (2GB).

As shown in Table 6.1, RL controllers with different width have almost equal computational time while changing the depth seems to have a clear effect. This becomes even more obvious when looking at Fig. 6.1 where the average for the depth/width is calculated from the values

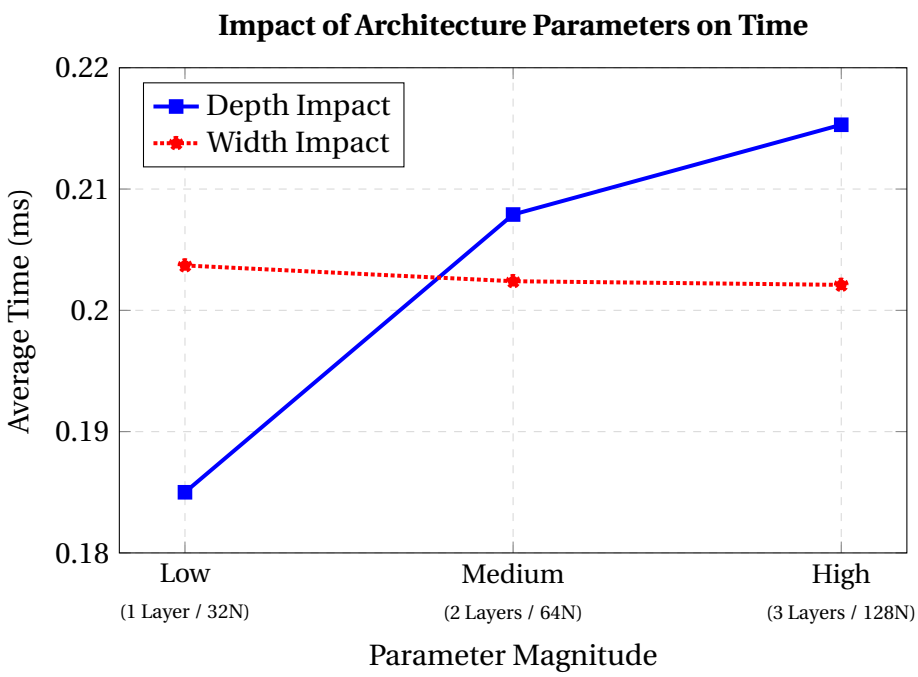


Figure 6.1: The Figure shows how the depth and width impact the average computational time of the agents.

in Table 6.1. The reason for this might be that neurons in the same layer can be computed in parallel, but layers have to be computed sequentially as they depend on the values in the previous layer.

Table 6.1 also shows that the different RL-agents are all significantly faster than the MPC variants, which would suggest that for real time application RL could be feasible in situations where MPC is not.

6.2 Case Study 1: How is Performance on the 1-DOF Levitating Ball Environment Affected by Model Parameters?

While computational time is important, tracking accuracy (RMSE) is the primary measure of control performance. This section analyzes the impact of network architecture (width and depth) on the RMSE for the 1-DOF levitating ball, culminating in a final comparison against the MPC baseline. The experiments were conducted as described by 5.

Impact of Network Architecture

A series of RL agents were trained with different network architectures, 1, 2 or 3 layers with 32, 64 or 128 neurons in each layer. Figure 6.2 to 6.19 and Table 6.2 to 6.19 compares their final

RMSE performance on the sinusoidal and sigmoid test trajectories. From these results we see that the overall best configuration is the 3-layers with 64 neurons. This might be because of the diverse training and the nonlinear constraints in form of the ground, top of the screen and the max force. The overall worst statistics are from the 1-layer networks, where the networks are too shallow for an efficient representation of the system dynamics. The 128-neuron architectures has a large overshoot while also rising slowly, which is an unusual combination. Performance might improve with extended training, even though it should have had plenty, or it is stuck in a local optimum.

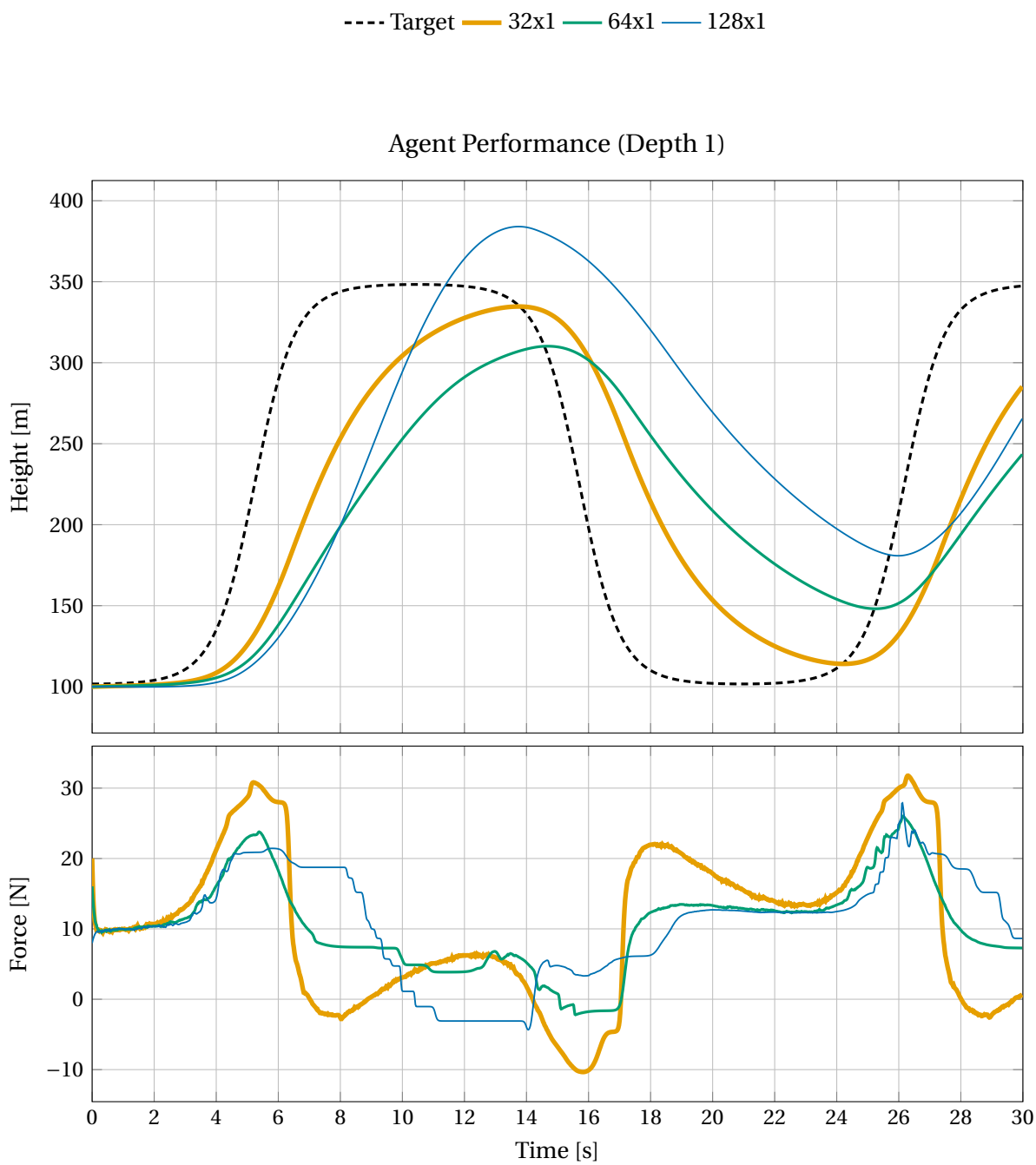


Figure 6.2: Performance of 1-layer architectures.

Controller	RMSE
ppo_test_arch[32]	70.396
ppo_test_arch[64]	95.376
ppo_test_arch[128]	114.401

Table 6.2: RMSE values for Sigmoid with $k = 5$ (Depth 1).

Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

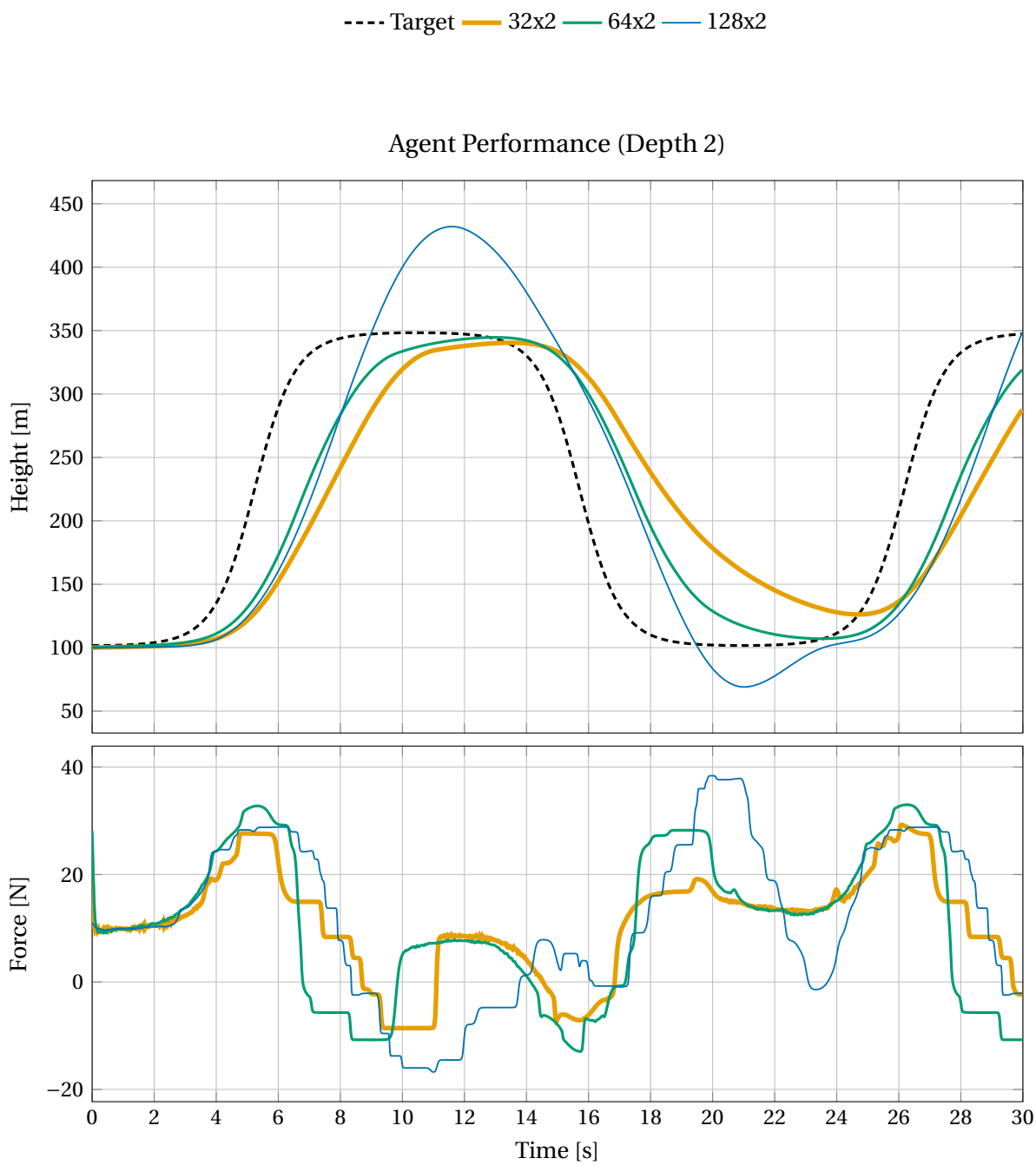


Figure 6.3: Performance of 2-layer architectures.

Controller	RMSE
ppo_test_arch[32, 32]	78.540
ppo_test_arch[64, 64]	58.960
ppo_test_arch[128, 128]	67.727

Table 6.3: RMSE values for Sigmoid with $k = 5$ (Depth 2).

Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

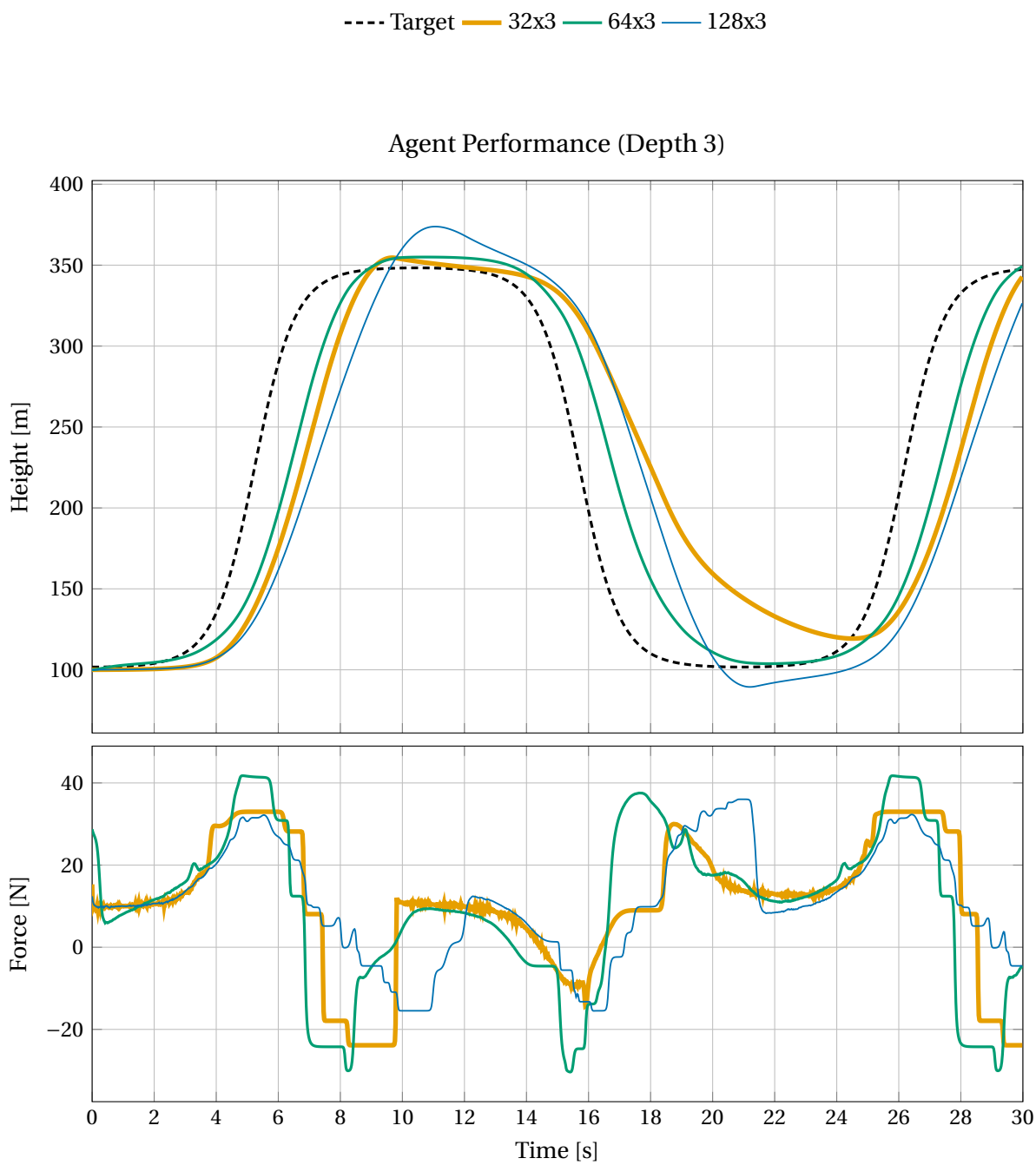
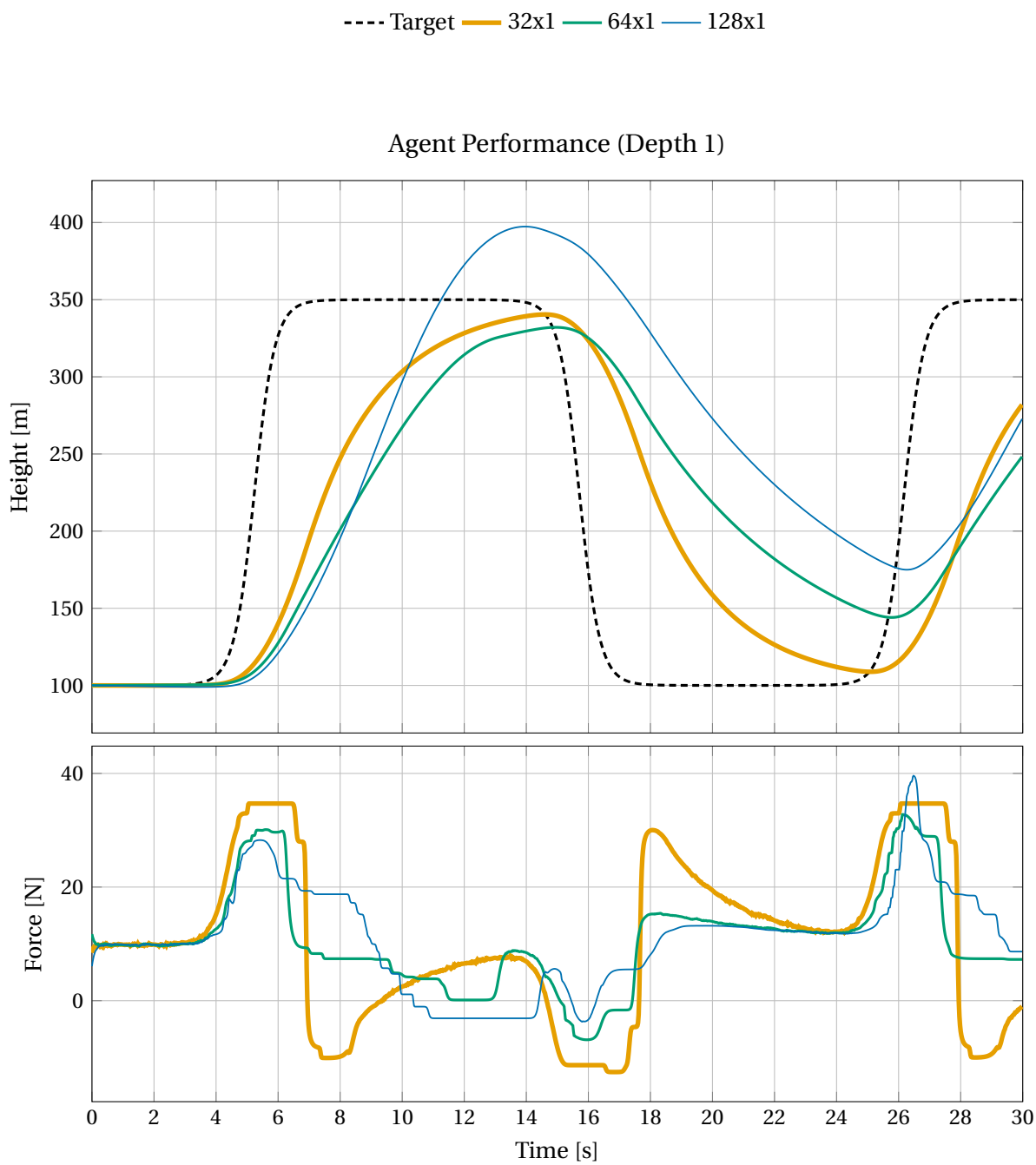


Figure 6.4: Performance of 3-layer architectures.

Controller	RMSE
ppo_test_arch[32, 32, 32]	62.937
ppo_test_arch[64, 64, 64]	40.598
ppo_test_arch[128, 128, 128]	66.343

Table 6.4: RMSE values for Sigmoid with $k = 5$ (Depth 3).

Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

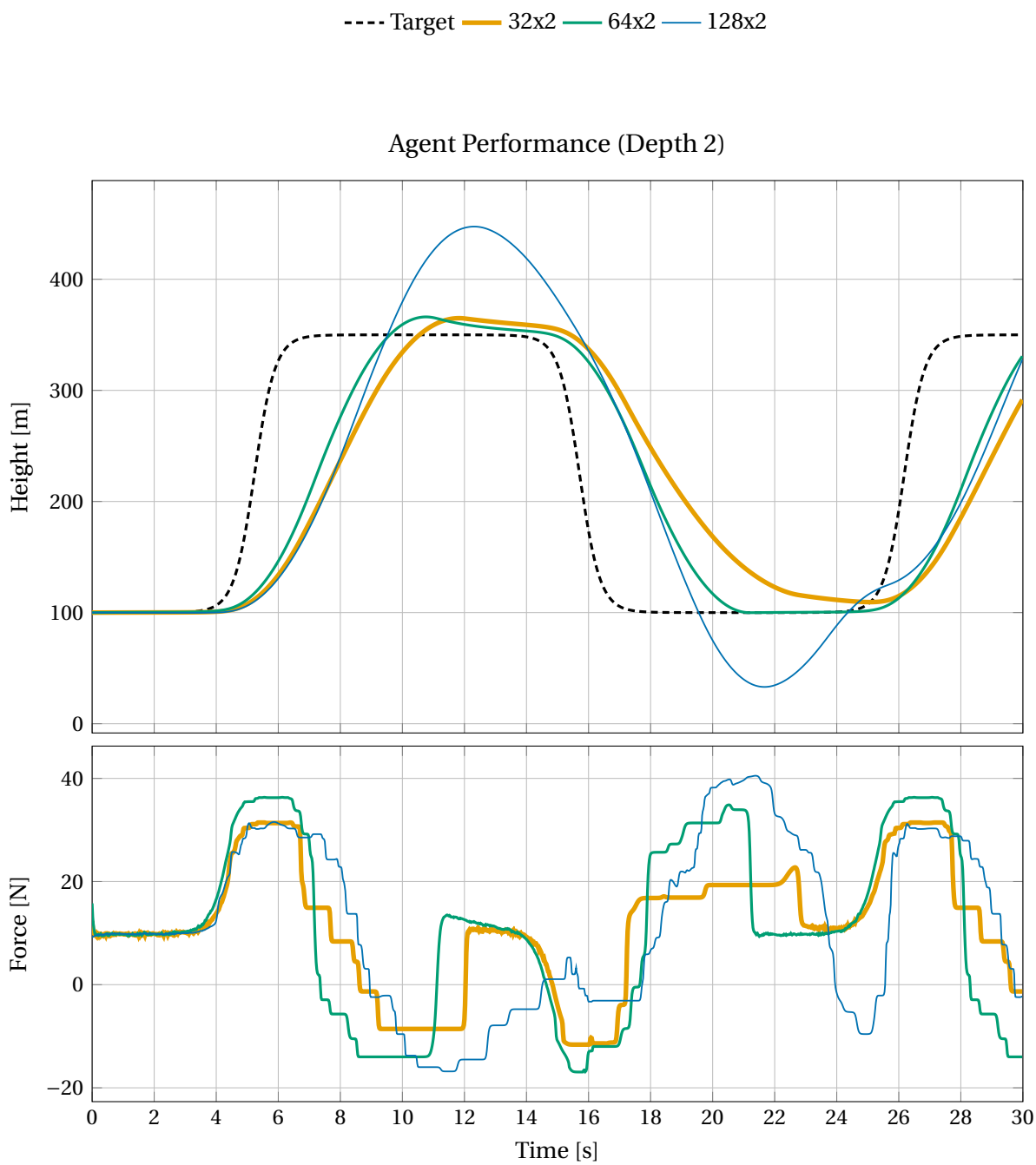


Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

Figure 6.5: Performance of 1-layer architectures.

Controller	RMSE
ppo_test_arch[32]	90.427
ppo_test_arch[64]	109.084
ppo_test_arch[128]	127.458

Table 6.5: RMSE values for Sigmoid with $k = 10$ (Depth 1).



Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

Figure 6.6: Performance of 2-layer architectures.

Controller	RMSE
ppo_test_arch[32, 32]	95.985
ppo_test_arch[64, 64]	81.193
ppo_test_arch[128, 128]	94.203

Table 6.6: RMSE values for Sigmoid with $k = 10$ (Depth 2).

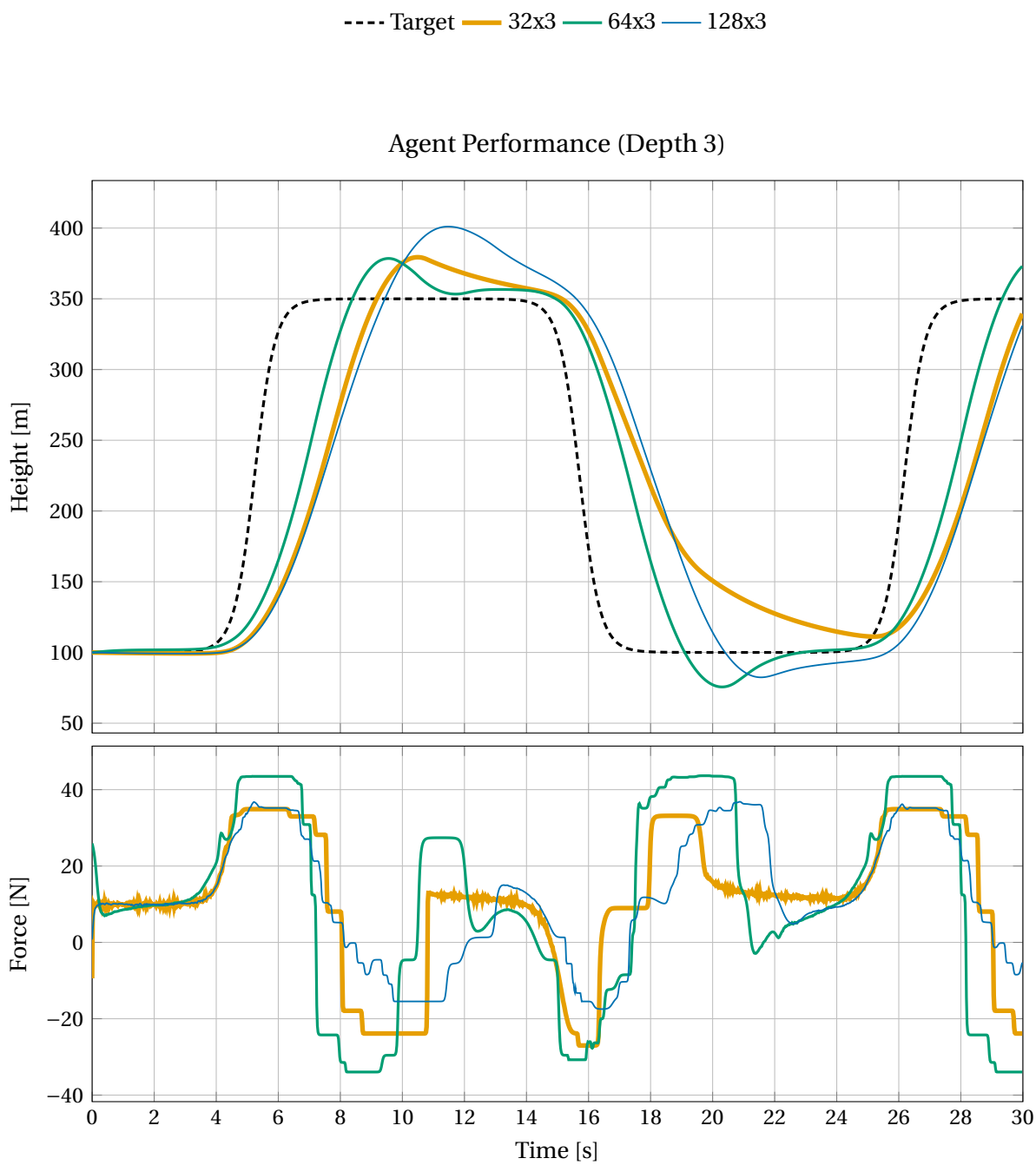
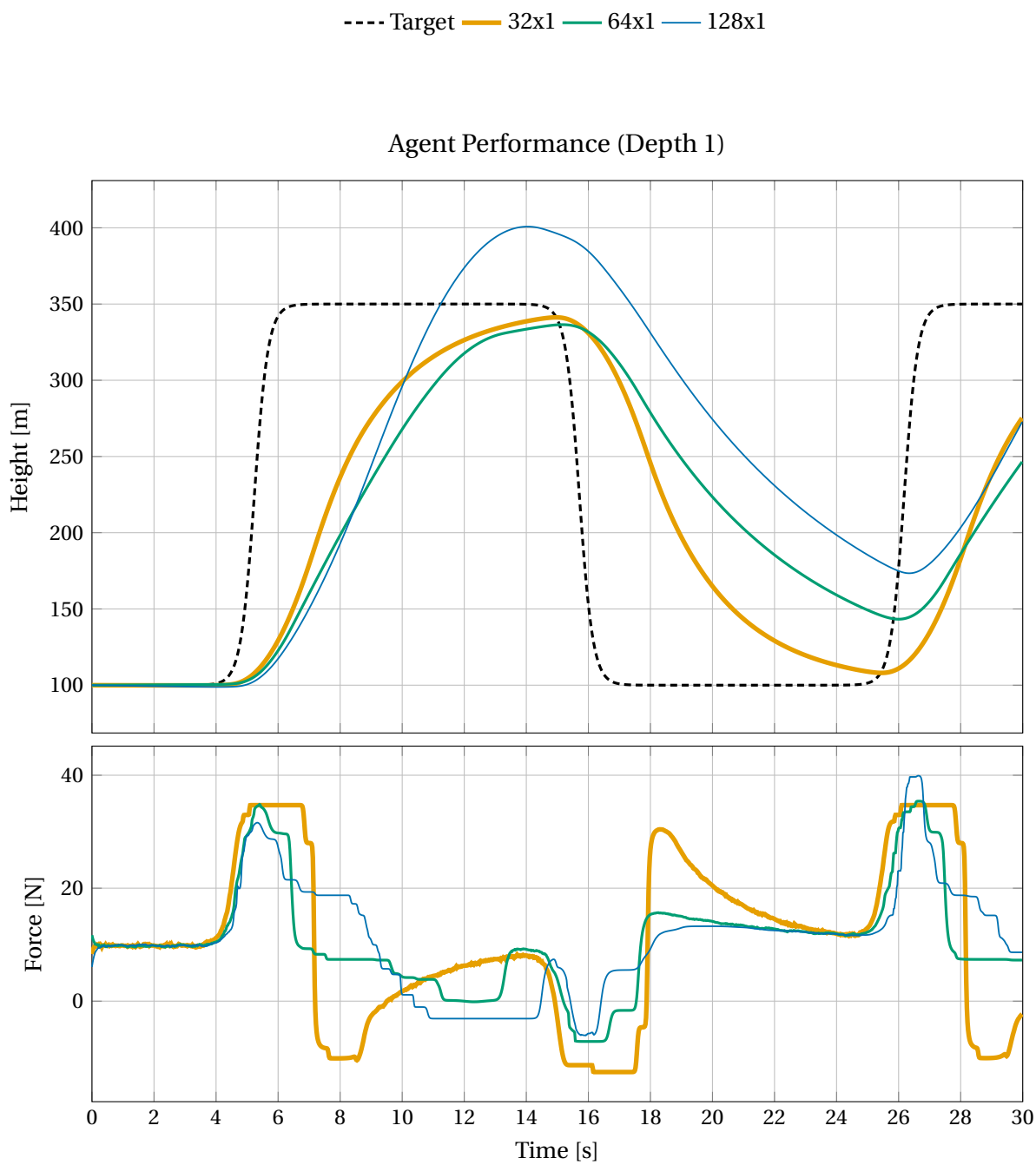


Figure 6.7: Performance of 3-layer architectures.

Controller	RMSE
ppo_test_arch[32, 32, 32]	84.084
ppo_test_arch[64, 64, 64]	66.093
ppo_test_arch[128, 128, 128]	89.480

Table 6.7: RMSE values for Sigmoid with $k = 10$ (Depth 3).

Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3



Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

Figure 6.8: Performance of 1-layer architectures.

Controller	RMSE
ppo_test_arch[32]	100.717
ppo_test_arch[64]	115.244
ppo_test_arch[128]	131.922

Table 6.8: RMSE values for Sigmoid with $k = 15$ (Depth 1).

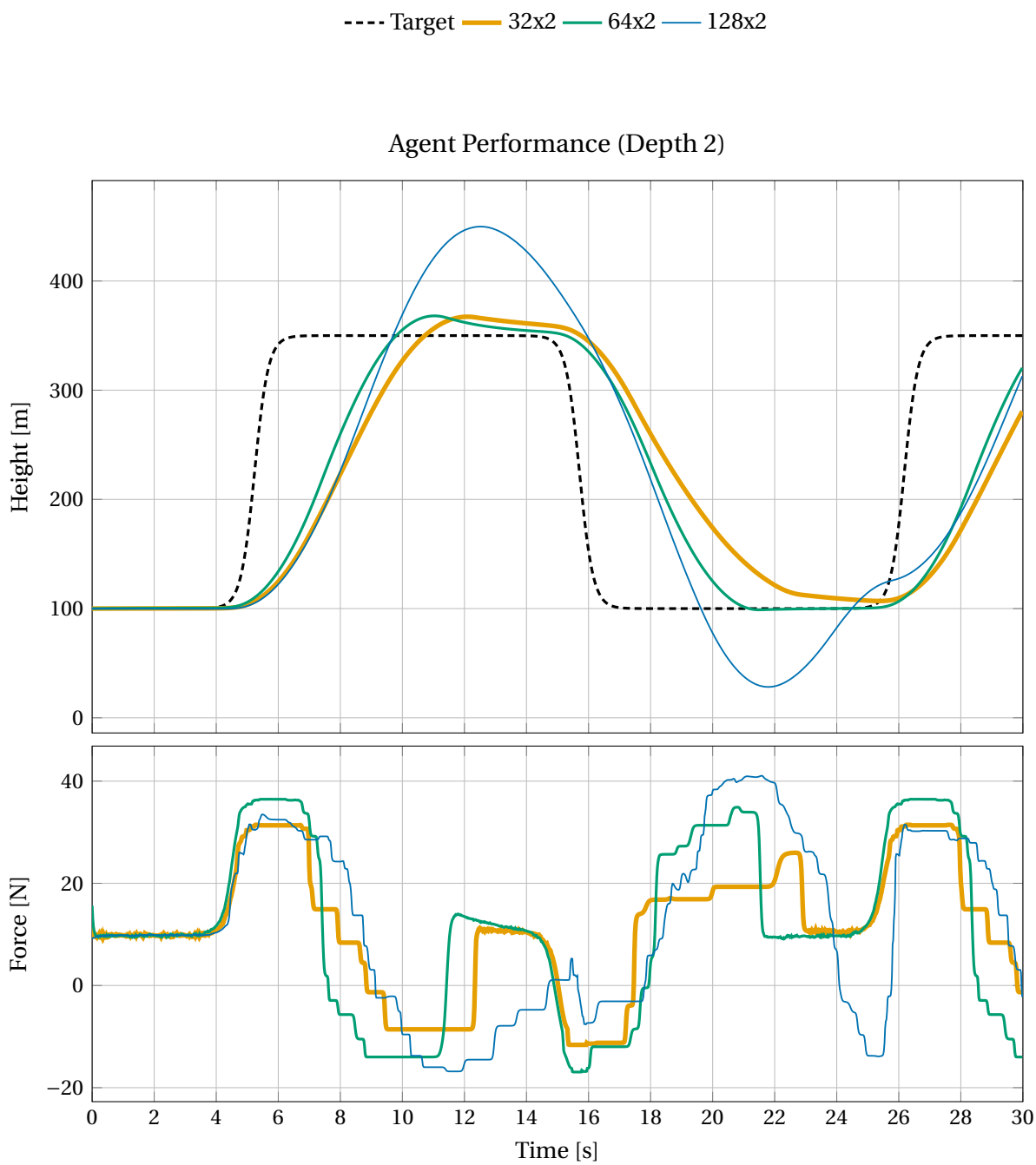


Figure 6.9: Performance of 2-layer architectures.

Controller	RMSE
ppo_test_arch[32, 32]	105.862
ppo_test_arch[64, 64]	93.032
ppo_test_arch[128, 128]	103.312

Table 6.9: RMSE values for Sigmoid with $k = 15$ (Depth 2).

Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

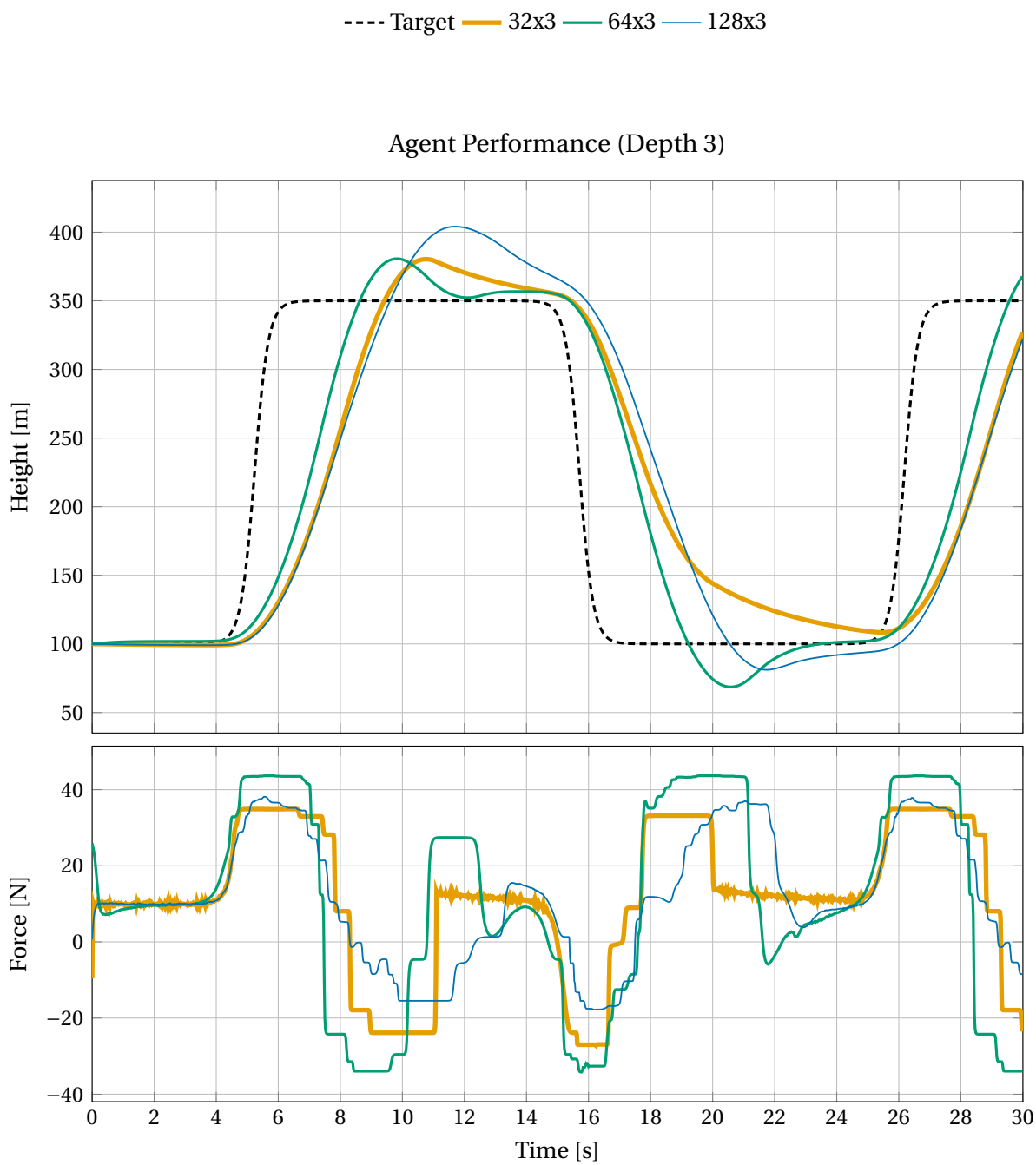


Figure 6.10: Performance of 3-layer architectures.

Controller	RMSE
ppo_test_arch[32, 32, 32]	93.519
ppo_test_arch[64, 64, 64]	79.158
ppo_test_arch[128, 128, 128]	99.536

Table 6.10: RMSE values for Sigmoid with $k = 15$ (Depth 3).

Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

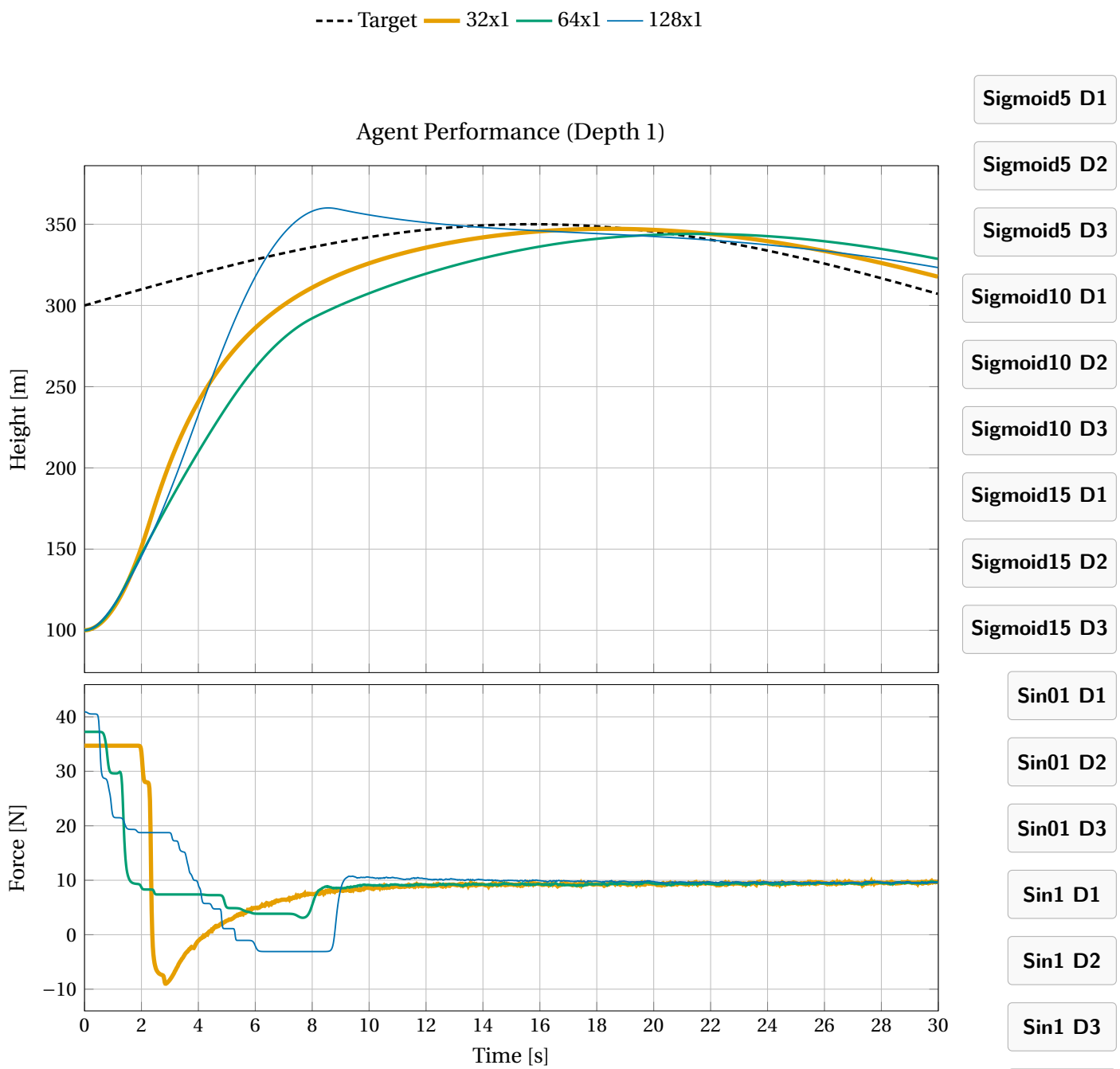
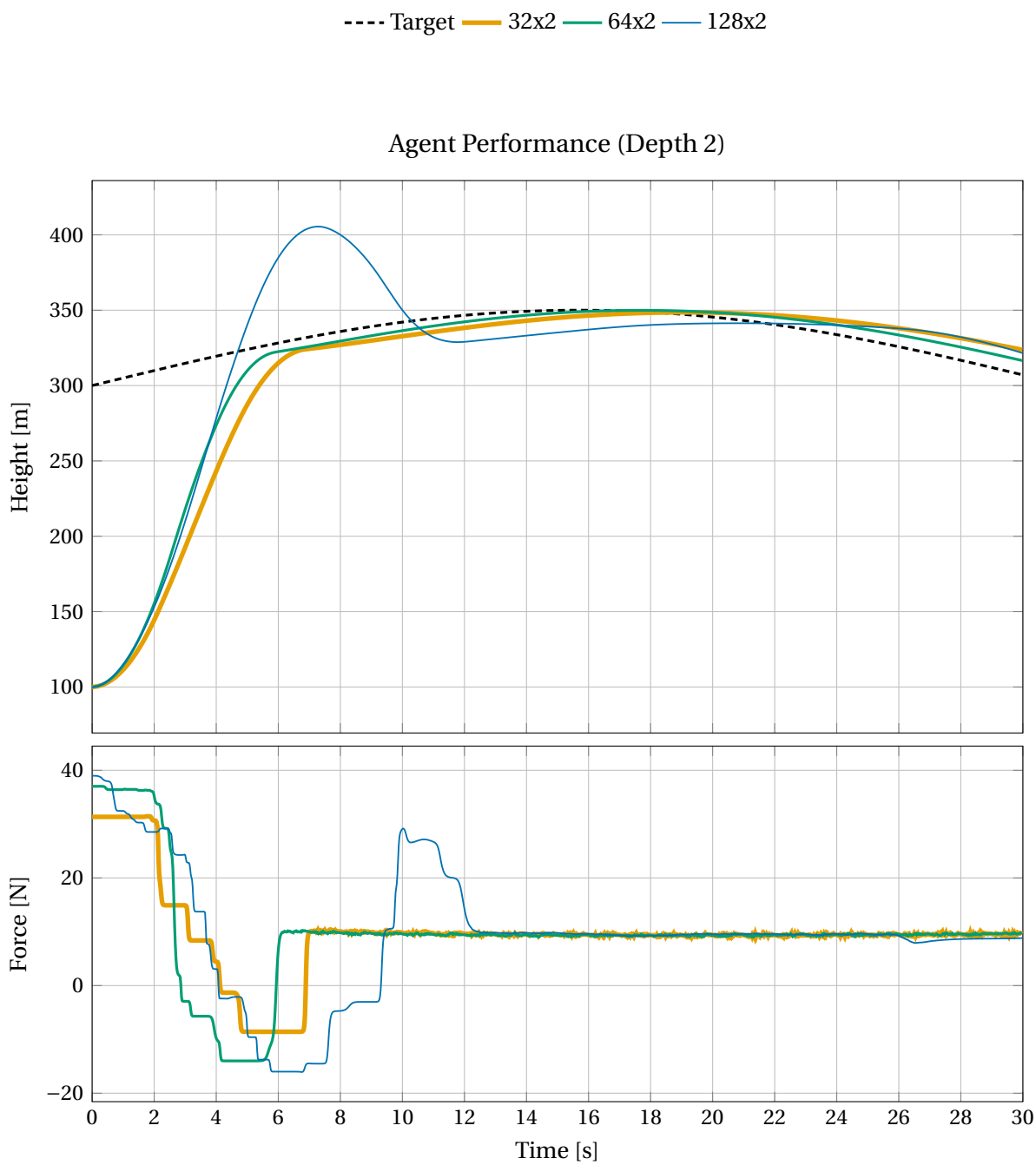


Figure 6.11: Performance of 1-layer architectures.

Controller	RMSE
ppo_test_arch[32]	60.380
ppo_test_arch[64]	67.811
ppo_test_arch[128]	61.149

Table 6.11: RMSE values for Sine with $\omega = 0.1$ (Depth 1).



Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

Figure 6.12: Performance of 2-layer architectures.

Controller	RMSE
ppo_test_arch[32, 32]	60.415
ppo_test_arch[64, 64]	55.742
ppo_test_arch[128, 128]	60.886

Table 6.12: RMSE values for Sine with $\omega = 0.1$ (Depth 2).

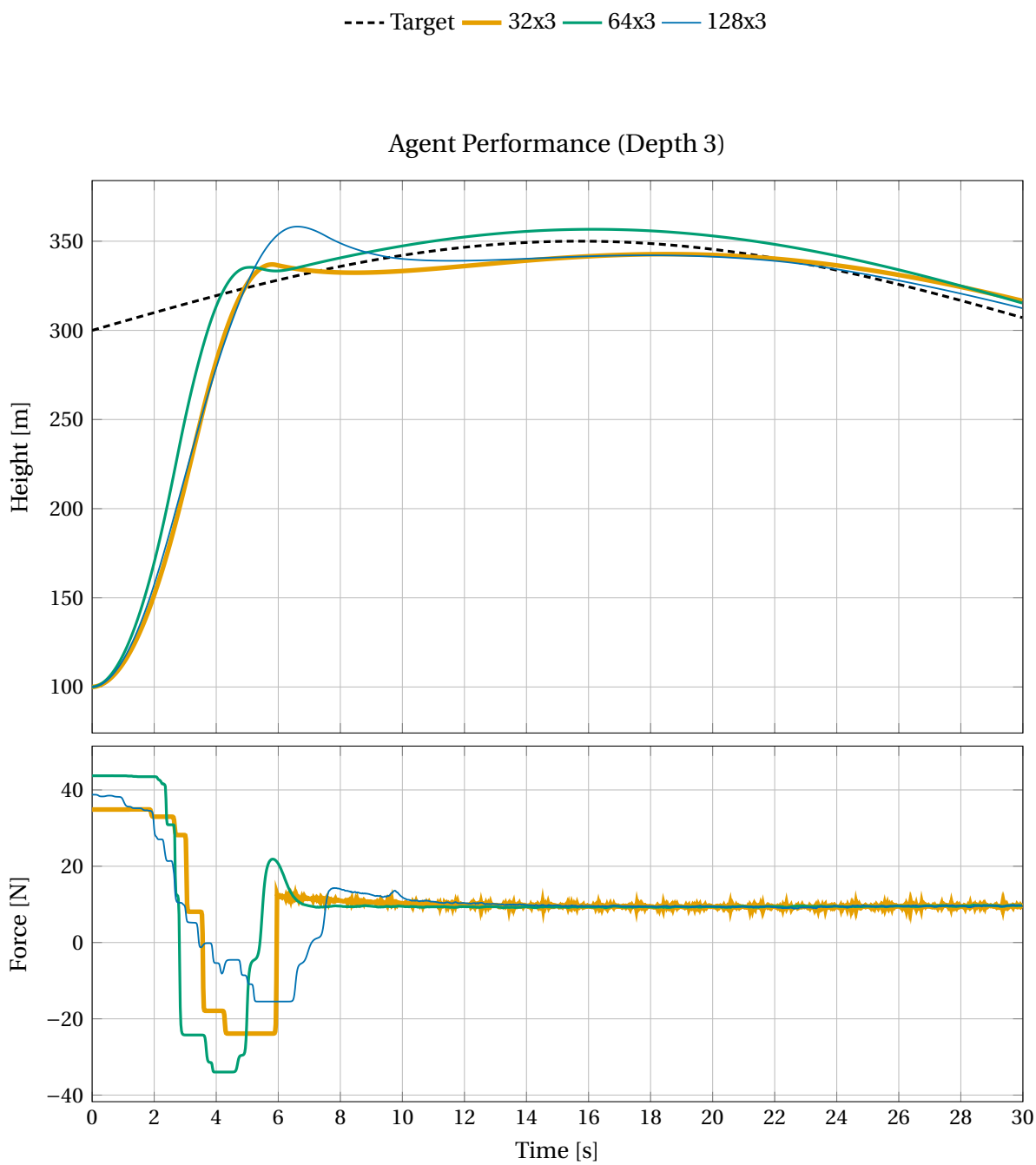
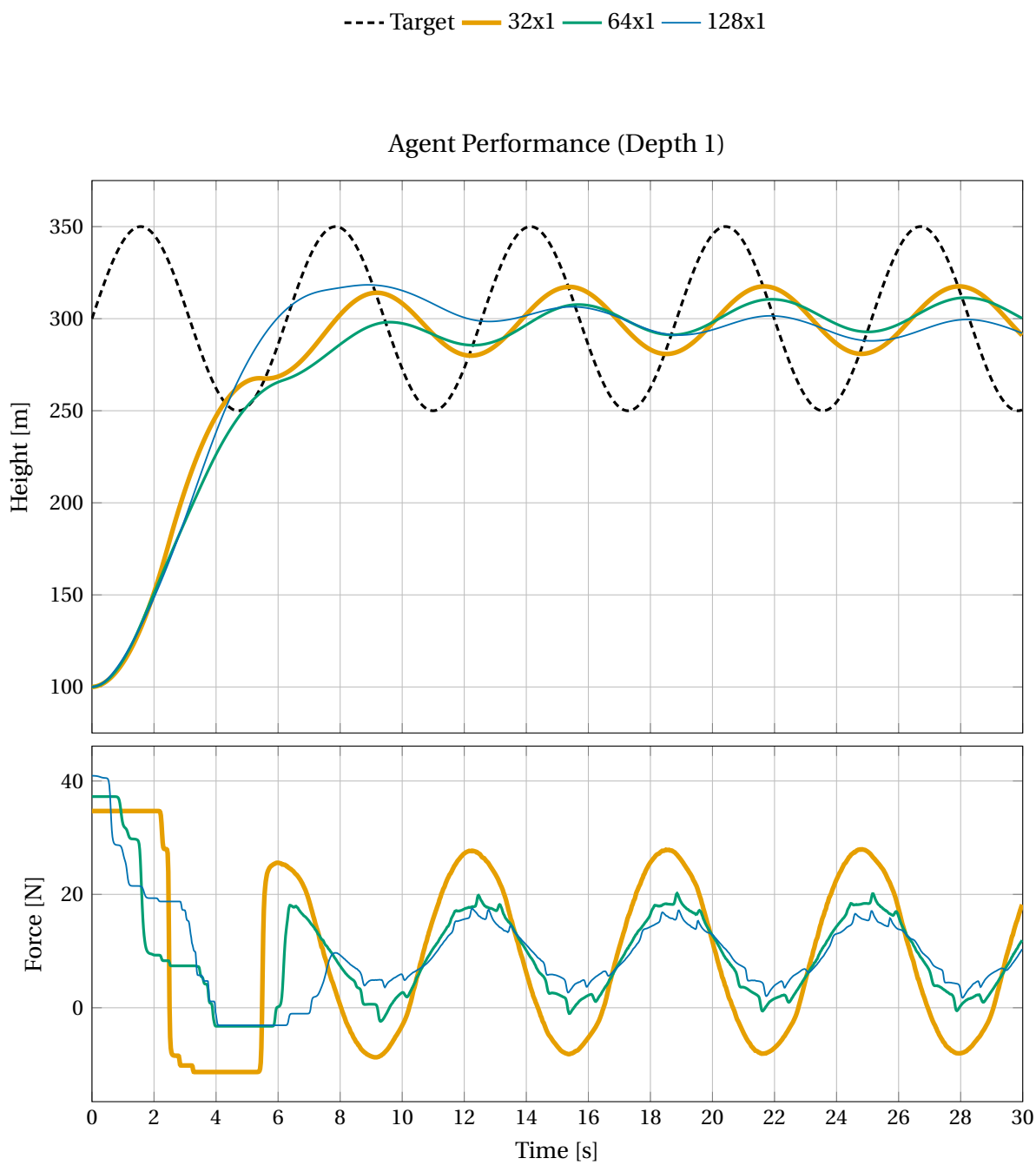


Figure 6.13: Performance of 3-layer architectures.

Controller	RMSE
ppo_test_arch[32, 32, 32]	56.320
ppo_test_arch[64, 64, 64]	51.832
ppo_test_arch[128, 128, 128]	55.614

Table 6.13: RMSE values for Sine with $\omega = 0.1$ (Depth 3).

Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

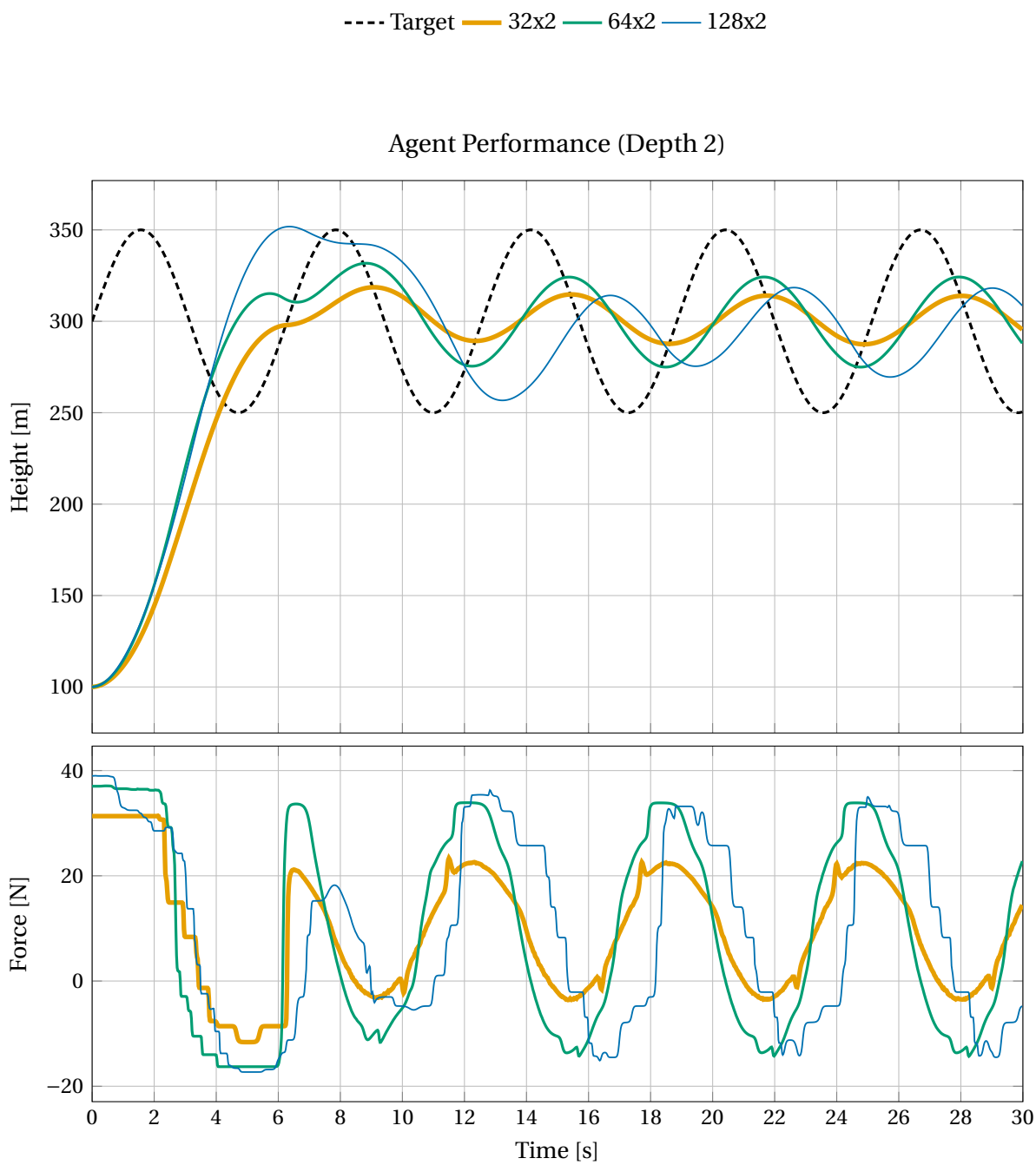


Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

Figure 6.14: Performance of 1-layer architectures.

Controller	RMSE
ppo_test_arch[32]	71.055
ppo_test_arch[64]	73.219
ppo_test_arch[128]	72.258

Table 6.14: RMSE values for Sine with $\omega = 1$ (Depth 1).



Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

Figure 6.15: Performance of 2-layer architectures.

Controller	RMSE
ppo_test_arch[32, 32]	72.453
ppo_test_arch[64, 64]	70.139
ppo_test_arch[128, 128]	77.964

Table 6.15: RMSE values for Sine with $\omega = 1$ (Depth 2).

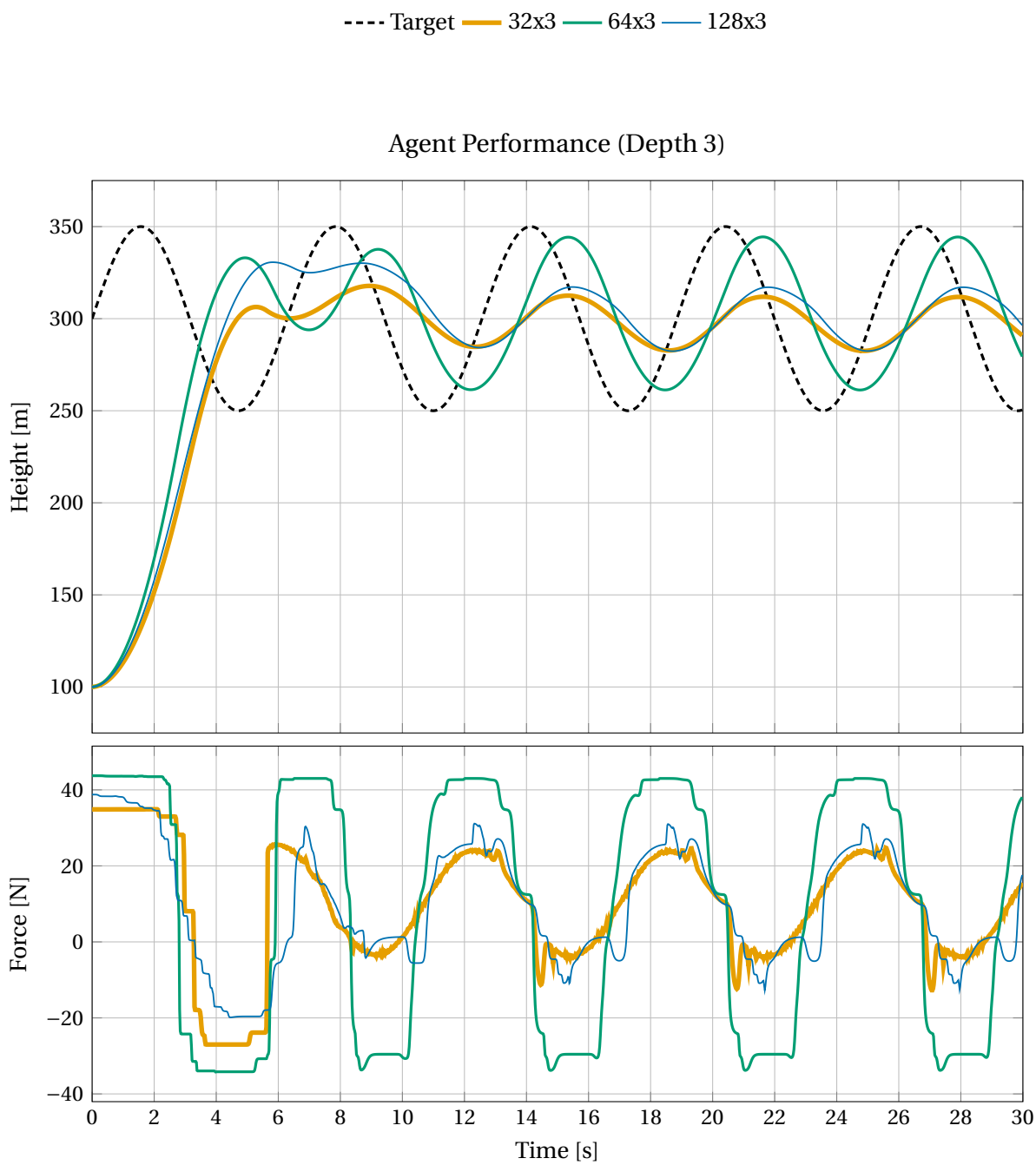


Figure 6.16: Performance of 3-layer architectures.

Controller	RMSE
ppo_test_arch[32, 32, 32]	70.829
ppo_test_arch[64, 64, 64]	70.522
ppo_test_arch[128, 128, 128]	71.385

Table 6.16: RMSE values for Sine with $\omega = 1$ (Depth 3).

Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

Figure 6.17: Performance of 1-layer architectures.

Controller	RMSE
ppo_test_arch[32]	66.053
ppo_test_arch[64]	69.860
ppo_test_arch[128]	68.939

Table 6.17: RMSE values for Sine with $\omega = 10$ (Depth 1).

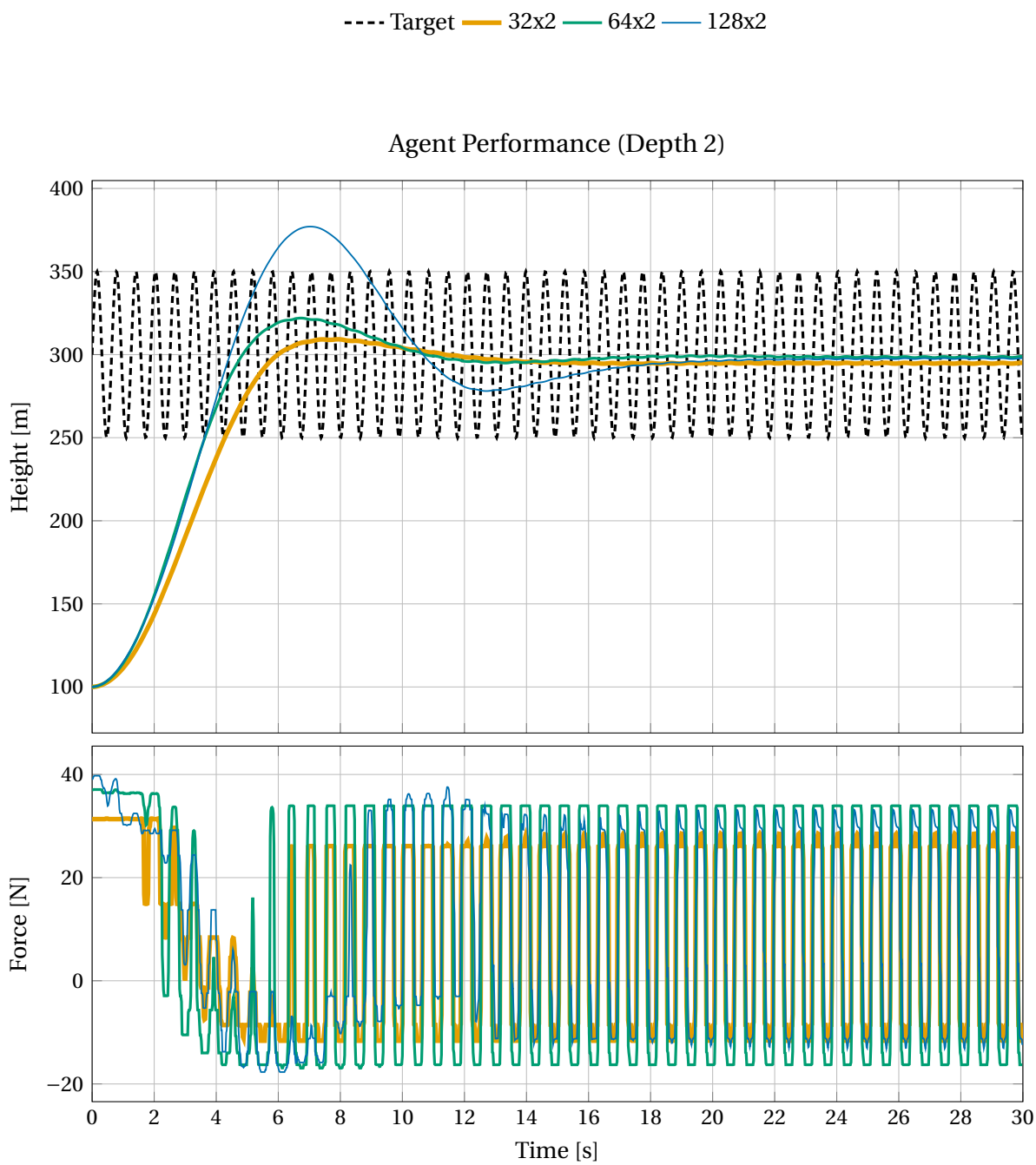
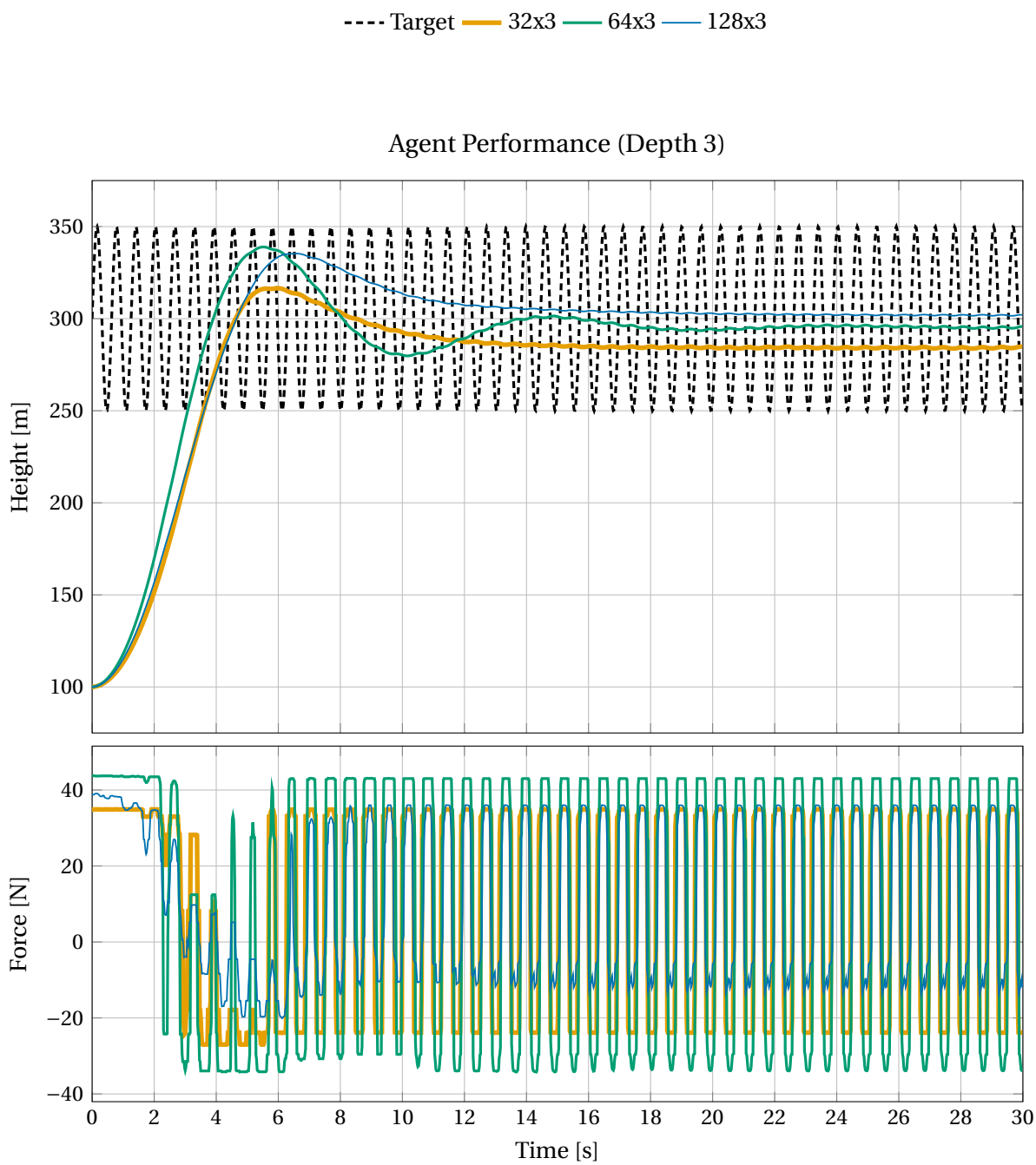


Figure 6.18: Performance of 2-layer architectures.

Controller	RMSE
ppo_test_arch[32, 32]	67.619
ppo_test_arch[64, 64]	64.676
ppo_test_arch[128, 128]	69.240

Table 6.18: RMSE values for Sine with $\omega = 10$ (Depth 2).

Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3



Sigmoid5 D1
Sigmoid5 D2
Sigmoid5 D3
Sigmoid10 D1
Sigmoid10 D2
Sigmoid10 D3
Sigmoid15 D1
Sigmoid15 D2
Sigmoid15 D3
Sin01 D1
Sin01 D2
Sin01 D3
Sin1 D1
Sin1 D2
Sin1 D3
Sin10 D1
Sin10 D2
Sin10 D3

Figure 6.19: Performance of 3-layer architectures.

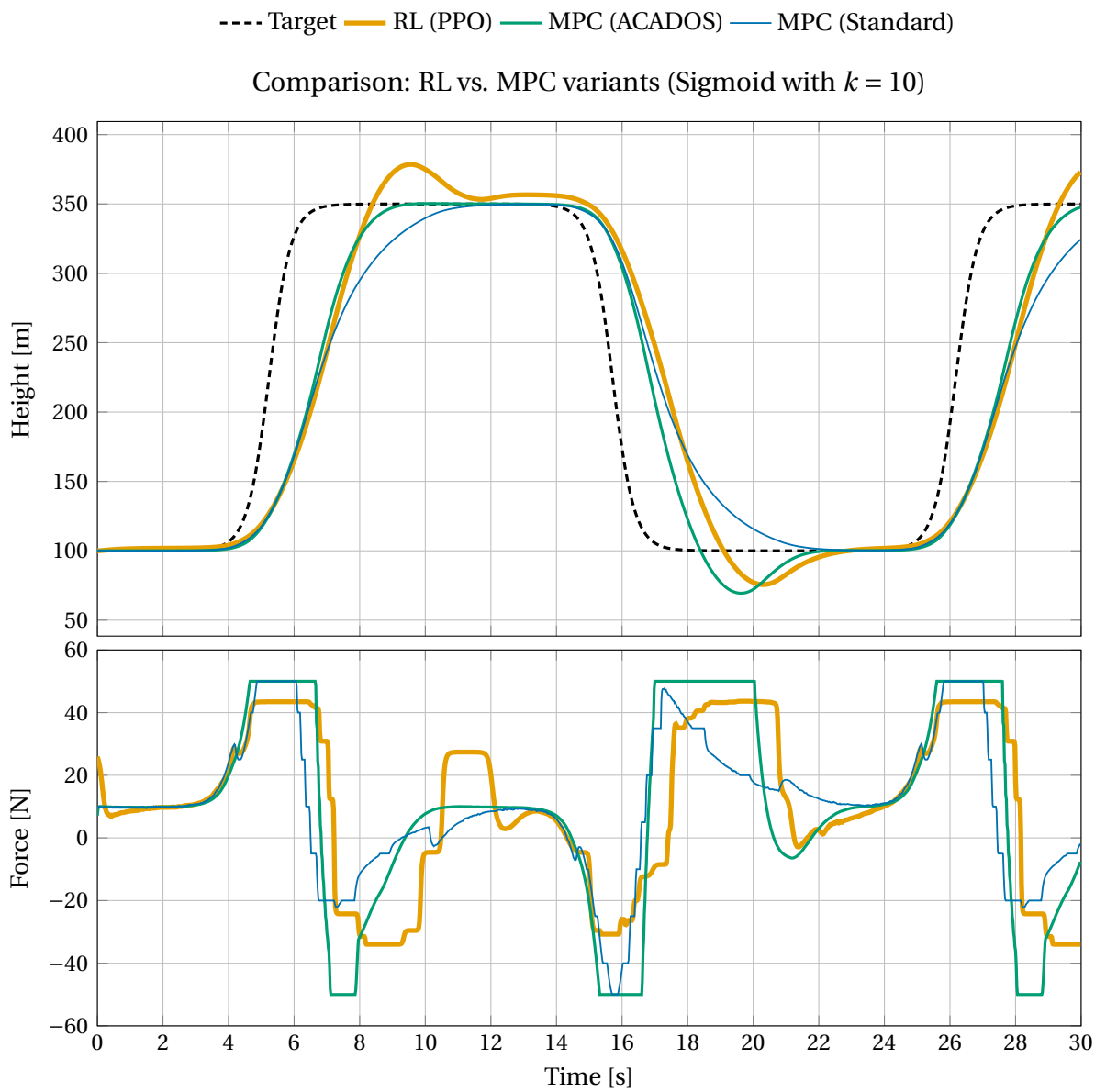
Controller	**RMSE**
ppo_test_arch[32, 32, 32]	65.978
ppo_test_arch[64, 64, 64]	62.708
ppo_test_arch[128, 128, 128]	65.043

Table 6.19: RMSE values for Sine with $\omega = 10$ (Depth 3).

When comparing the 3-layer 64-neuron agent to an `acados` MPC and to a Standard MPC we see from Fig.6.20 that it performs slightly worse. The two MPCs have perfect knowledge of the environment and it is not surprising that they perform at such a high level in this simple scenario. This shows that RL-agents can achieve performance comparable to MPCs.

Impact of Model Mismatch

For model mismatch the 2-layer 64-neuron model was used and it was tested on cases where the mass of the ball was changed from $1.0kg$ to $0.3kg$ and $1.7kg$. The light ball accelerates more with the same applied force, which makes it respond faster to control action. This makes it track the target better. The opposite is true for the heavier ball, it is harder to accelerate and the overshoots get bigger. This can be seen in Fig. 6.21 and 6.22.



Controller	RMSE [m]
RL (PPO)	66.093
MPC (Standard)	65.404
MPC (acados)	59.294

Figure 6.20: Performance comparison of RL against two MPC formulations for tracking the Sigmoid reference with $k = 10$. The table shows root mean square error (RMSE) over the 30s trajectory.

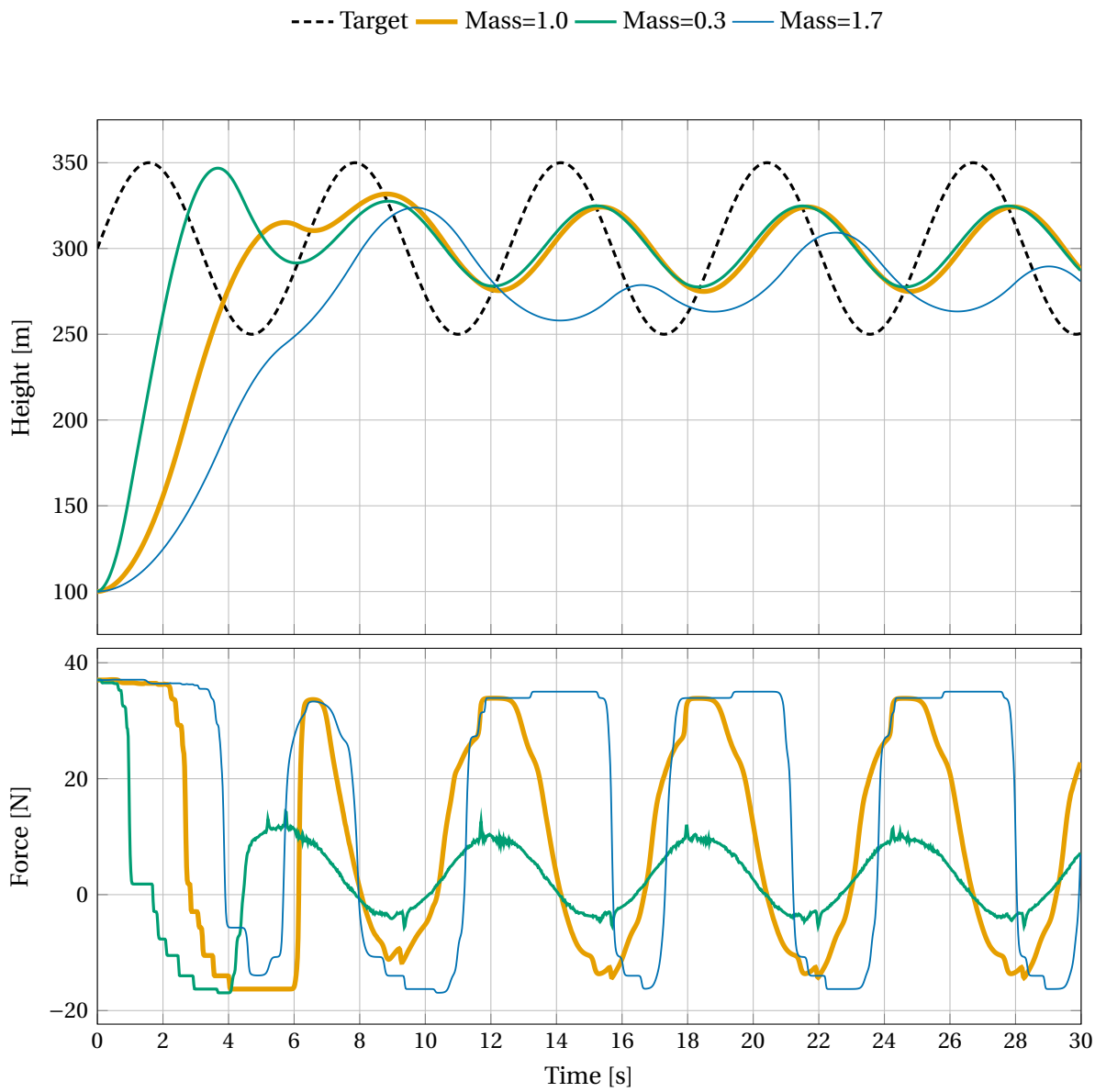


Figure 6.21: Robustness comparison for Sine with $\omega = 1$.

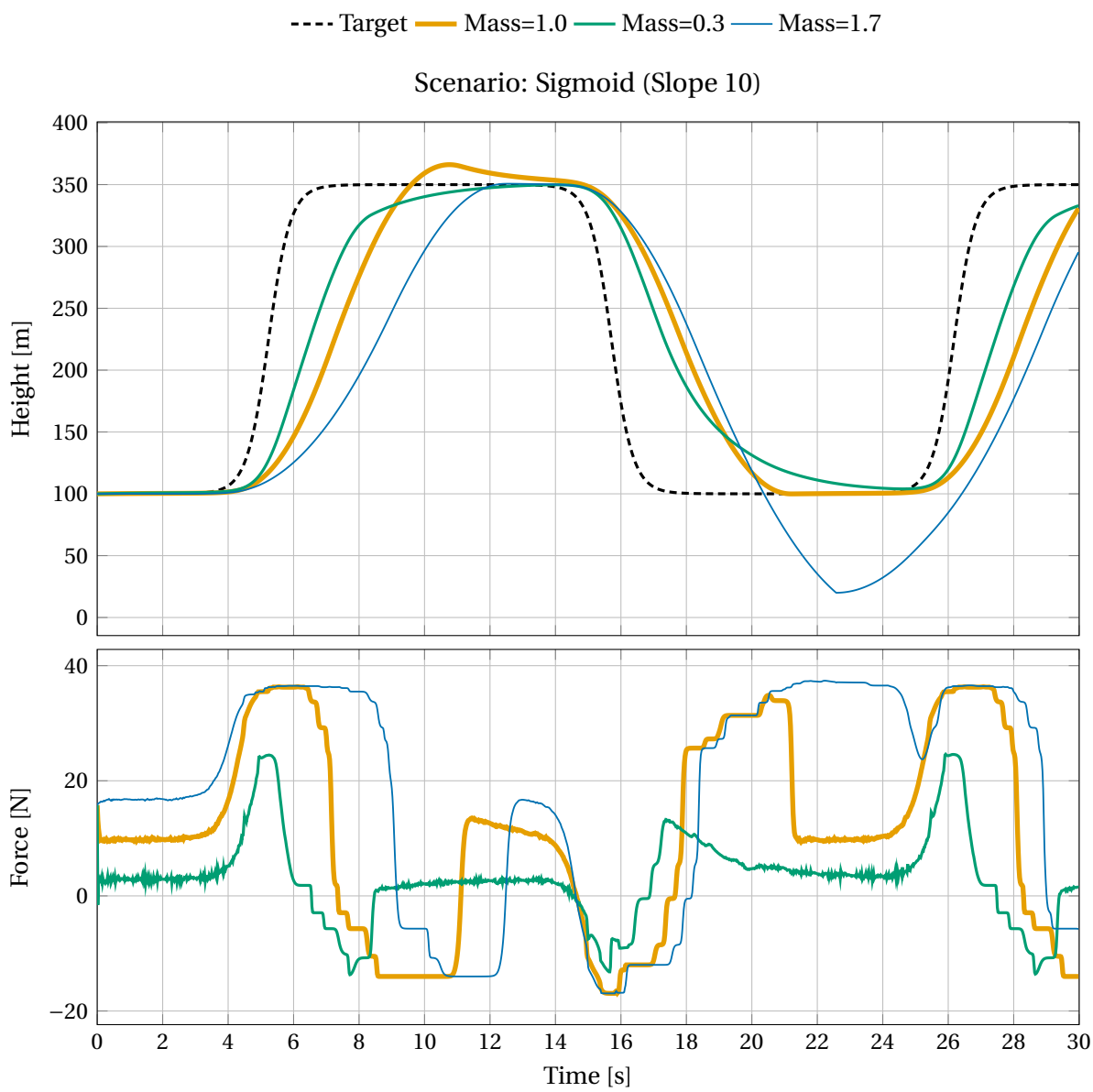


Figure 6.22: Robustness comparison for Sigmoid with $k = 10$.

6.3 Case Study 2: In Which Circumstances is RL Able to Stabilize the Magnetic Levitation (Maglev) Simulator

The MagLev system presents a significantly more complex control challenge than linear systems due to its inherently unstable and nonlinear dynamics. This section investigates the specific boundary conditions and architectural requirements necessary for a RL agent to successfully stabilize the levitating magnet. The results can be seen in Fig. 6.23 and 6.24.

Two distinct control architectures described in 5 were used:

1. **Hybrid Control (RL + LQR)**.

2. **End-to-End Control (Pure RL)**

The models were trained on the simulator provided by the Github for the project [2], with the reward function seen in Eq. 6.1, which prioritizes "survival", but has stability at the equilibrium as a secondary objective. This proved to give the critic a good way of assessing good versus bad actions, so that the actor could learn a good policy. Here the exponentials were used to keep the values bounded. This reward function is a good place to start when developing this further.

$$R_t = \begin{cases} -100 & \text{if } \mathbf{s}_{t+1} \in \mathscr{S}_{\text{fail}} \\ \underbrace{1}_{\text{Survival}} + \underbrace{\exp\left(-100\|\mathbf{p}_t - \mathbf{p}_{\text{target}}\|^2\right)}_{\text{Position}} + \underbrace{0.1\exp\left(-10\|\mathbf{v}_t\|^2\right)}_{\text{Velocity}} + \underbrace{0.1\exp\left(-\|\boldsymbol{\theta}_{xy,t}\|^2\right)}_{\text{Orientation}} & \text{otherwise} \end{cases} \tag{6.1}$$

6.3.1 Architectural Complexity and The Stability-Exploration Trade-off

A fundamental challenge observed in both the Hybrid and End-to-End approaches was balancing system stability with the stochasticity required for learningthe exploration-exploitation dilemma. In the context of the unstable Maglev dynamics, high variance in action sampling (essential for exploring new policy spaces) frequently led to immediate system failure (crashing the magnet) before the agent could gather meaningful gradient information. Conversely, a low standard deviation in the action distribution maintained stability for a bit longer, but often resulted in no exploration and consequently no learning.

The structural requirements for the neural networks differed significantly between the two approaches. The Hybrid (RL + LQR) configuration functioned effectively with a smaller network architecture (128×2 hidden layers). In this setup, the LQR controller provided baseline stability, effectively reducing the RL agent's role to a "residual learner" or fine-tuner. The primary objective of the Hybrid agent was effectively to "do no harm" while optimizing secondary objectives.

In contrast, the End-to-End RL agent required a deeper and wider network capacity (256×2 hidden layers) to capture the full nonlinear dynamics of the system. Without the stabilizing influence of the LQR, the network required increased expressivity to approximate the complete control law necessary to prevent divergence.

6.3.2 Temporal Resolution and Real-Time Constraints

One of the most significant findings of this case study was the disparity in the required simulation frequencies (control loop rates) between the two architectures.

Table 6.20: Comparison of Control Architectures and Requirements

Parameter	**Hybrid (RL + LQR)**	**Pure RL (End-to-End)**
Network Architecture	128×2	256×2
Required Frequency	20 Hz	1000 Hz
Primary Role	Augmentation/Fine-tuning	Full Stabilization
Stability Baseline	Provided by LQR	Learned from scratch

As detailed in Table 6.20, the Pure RL solution necessitated a simulation frequency of 1000 Hz to function effectively. Both 20 Hz and 100Hz were also tested, but did not lead to satisfactory results. Other frequencies between 100 Hz and 1000 Hz might work, but were not tested. The unstable dynamics of the MagLev system require rapid correction; at lower frequencies, the state diverges too far between control actions for the RL agent to recover. Conversely, the Hybrid system remained stable at a significantly lower frequency of 20 Hz. This is most likely due to the LQR having a linearized model of the system from the start, which makes it choose a good action every time. The high required frequency imposes strict real-time constraints on the deployment of Pure RL solutions for MagLev, requiring hardware capable of sub-millisecond inference times.

6.3.3 Hyperparameter Sensitivity Analysis

Achieving a policy capable of stabilizing the magnetprior to any performance optimizationproved to be the primary hurdle. Sensitivity analysis revealed that successful stabilization was contingent upon a narrow range of hyperparameters.

- **State Space Normalization:** Rigorous normalization of the state space was found to be a prerequisite for convergence. Without it the agent struggled as the difference between velocities can be quite high while the positions and angles typically have very small values.
- **Action Distribution (Standard Deviation):** As noted in the stability trade-off, the initialization of the log standard deviation in the actor network was critical. An initialization that was too high caused immediate instability, while too low prevented the discovery of a stabilizing manifold.
- **Batch Size and Learning Rate:** The actor's mini-batch size showed a high correlation with stability. Larger batch sizes generally provided more stable gradient estimates, smoothing out the noise inherent in the unstable trajectories at the cost of either slow or no learning.

The results from the hyperparameter experimentation, seen in Listing 6.1, gave a convergent and fairly stable learning.

Ultimately, the transition from Hybrid to Pure RL demonstrated that while RL is capable of stabilizing highly nonlinear systems, it requires an increase in computational resources (network size) and temporal resolution (sampling frequency) compared to classical or hybrid methods. Notably, the pure RL actions varies a lot, which might not be feasible in the physical system. It is also not perfect, starting in the equilibrium leads to oscillations and it seems to have found a policy where making the magnet hover over the target is desirable. In Fig. 6.24 γ slowly drifts away, which is expected as it is not considered in the reward function. γ is the orientation about the z-axis, and since the magnet is a disk, the orientation does not matter to the dynamics.

```
stdBias = -0.43;
stdPath = [
    fullyConnectedLayer(4, 'Name', 'StdDevFC', ...
                        'WeightsInitializer', 'zeros', ...
                        'Bias', ones(4, 1) * stdBias)
    softplusLayer('Name','ActionStdDev')
];

agentOpts = rlPPOAgentOptions(...
    'SampleTime', Ts,...
    'ExperienceHorizon', 2048,...
    'ClipFactor', 0.15,...
    'EntropyLossWeight', 0.01,...
    'MiniBatchSize', 256,...
    'NumEpoch', 3,...
    'AdvantageEstimateMethod', 'gae',...
    'GAEFactor', 0.95,...
    'DiscountFactor', 0.995, ...
    'NormalizedAdvantage', 'current');
agentOpts.ActorOptimizerOptions.LearnRate = 3e-4;
agentOpts.CriticOptimizerOptions.LearnRate = 3e-4;
```

Listing 6.1: PPO Agent Configuration and Standard Deviation Layer

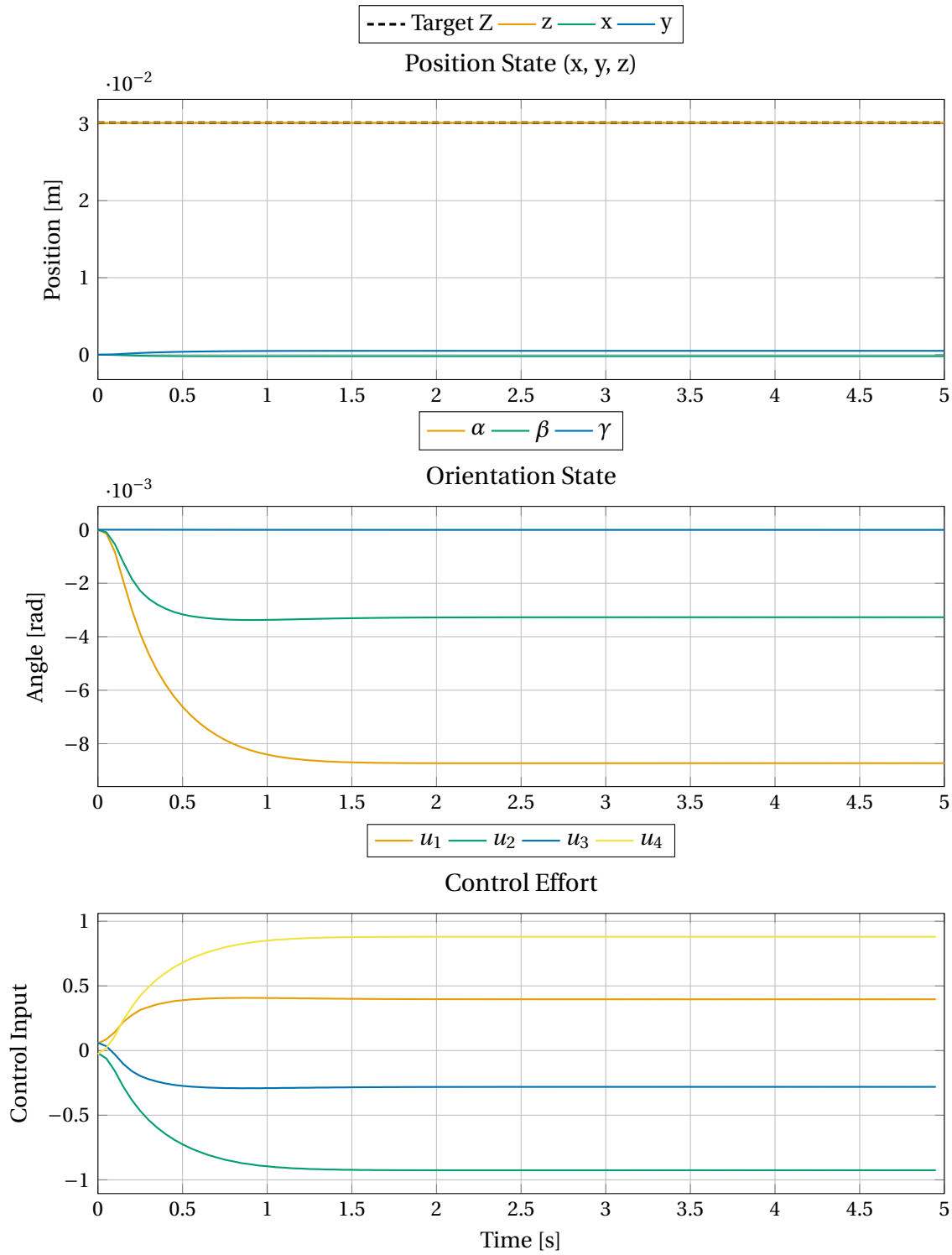


Figure 6.23: Simulation results showing Position, Orientation, and Control Effort over time with a RL+LQR controller.

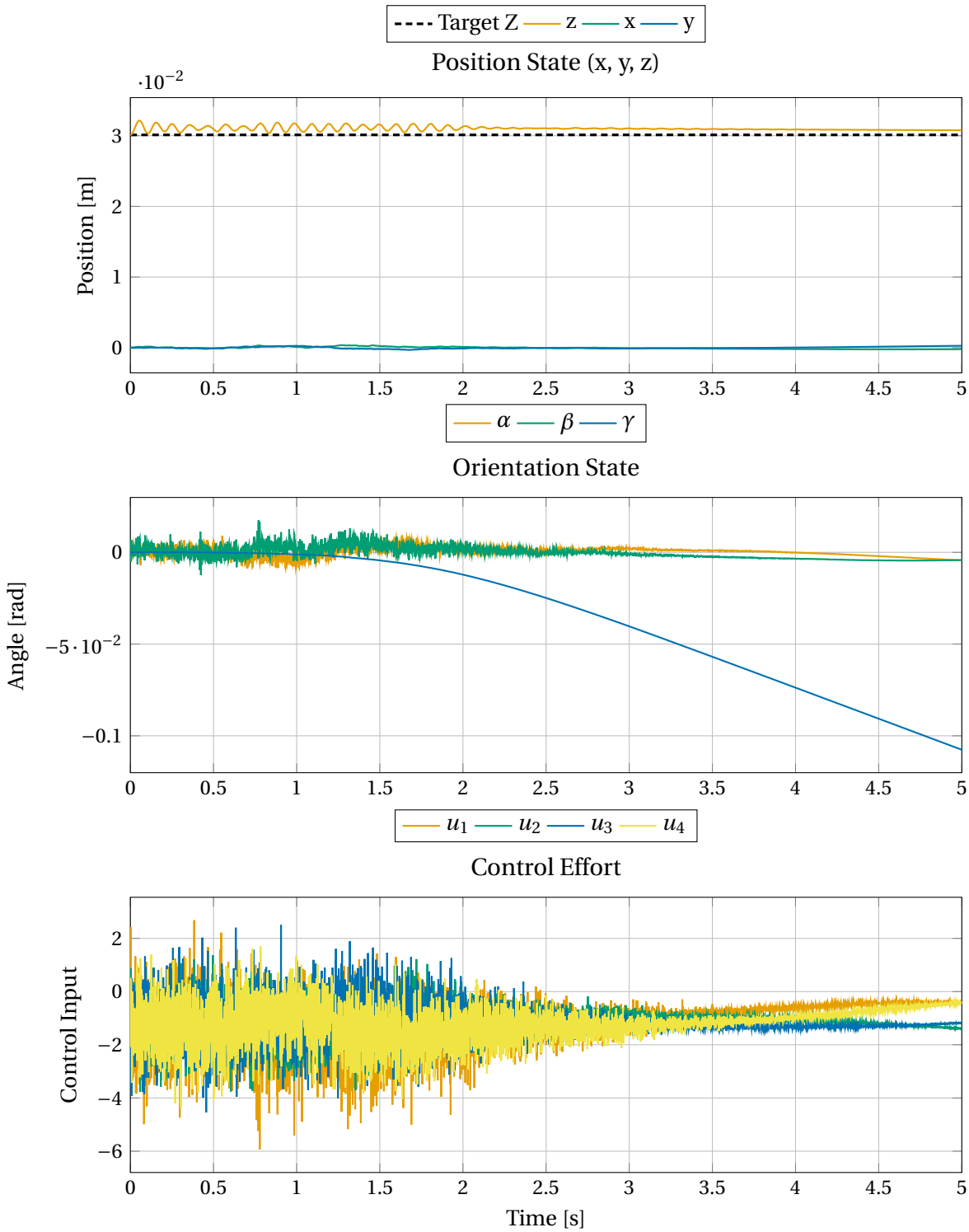


Figure 6.24: Simulation results showing Position, Orientation, and Control Effort over time with a pure RL controller.

Chapter 7

Conclusions, Discussion, and Recommendations for Further Work

This chapter summarizes the findings of the project, discusses their broader implications regarding the feasibility of RL for MagLev, and outlines a roadmap for future research and development.

7.1 Summary and Conclusions

The primary goal of this thesis was to investigate the limits and feasibility of applying RL for the control of a MagLev system, specifically the "Maggy" platform. This involved developing a training pipeline, comparing RL against a classical MPC baseline, and achieving stable levitation in a 6-DOF simulation.

Regarding the specific objectives defined in Section 1.2:

1. **Quantitatively compare RL with MPC across a set of metrics and environments:**
   This objective was achieved through the 1-DOF Levitating Ball case study. The results demonstrated that RL agents (specifically PPO) could achieve tracking performance comparable to the MPC baseline, particularly when utilizing 3-layer network architectures. In terms of computational efficiency, the RL agents generally outperformed the MPC baseline, with inference times averaging between 0.18ms and 0.22ms depending on network depth. This confirms that while training is computationally expensive, the resulting RL policy is highly efficient for real-time execution.

2. **Stabilizing a levitating magnet with RL:**
   This objective was met in the simulation environment. Two distinct control strategies were successfully implemented:

- A **Hybrid Architecture (RL + LQR)**, where the RL agent augmented a linear controller. This proved to be robust and capable of operating at a lower control frequency (20 Hz).
- A **Pure RL (End-to-End)** solution, which successfully stabilized the system from scratch. However, this approach required a significantly higher control frequency (1000 Hz) and a larger NN capacity to handle the fast, unstable dynamics of the MagLev plant.

7.2 Discussion

The findings of this project highlight both the potential and the distinct challenges of applying "model-free" learning to inherently unstable physical systems.

The Stability-Exploration Trade-off: A recurring theme throughout the experiments was the difficulty of balancing exploration with stability. In the 6-DOF environment, the margin for error is negligible; a random exploratory action often leads to an unrecoverable state (crashing the magnet) before the agent can learn from it. This explains why the Hybrid (RL+LQR) approach was significantly easier to trainthe LQR provided a "safety net" that kept the system within a recoverable region, allowing the RL agent to explore without constantly terminating the episode.

Real-Time Feasibility and Frequency Requirements: Perhaps the most critical finding for future implementation is the disparity in frequency requirements. The Pure RL agent requires a 1000 Hz control loop to maintain stability. While the target hardware supports up to 2000 Hz (a 0.5 ms cycle), a major concern is whether this budget is sufficient to accommodate both the RL inference and the state estimator. If the estimator is computationally expensive, it may not be feasible to run the full loop at 2000 Hz, or even 1000 Hz. This finding implies that the bottleneck for the Pure RL approach is likely not the neural network inference time itself, as that was measured to be around 0.2 ms, but the remaining 0.8 ms available for state estimation within the tight real-time window.

Network Architecture Sensitivity: The parameter sweeps in the 1-DOF environment revealed that "bigger" is not always "better." The 128-neuron networks frequently underperformed compared to the other architectures, likely due to overfitting or optimization difficulties given the simplicity of the system. This underperformance highlights the risks of over-parameterization in simple environments. It suggests that architectural selection should prioritize matching model capacity to task complexity, as compact networks are not only more efficient for embedded control but, as shown here, can fundamentally outperform their larger counterparts.

7.3 Lessons Learned

Reflecting on the development process, several key insights and "mistakes" provided valuable learning opportunities:

- **Oversimplification of the 1-DOF Simulator:** The 1-DOF simulator lacked sufficient complexity to fully capture the nuances of the target system. Incorporating a static permanent magnet alongside the controllable electromagnet would have introduced realistic non-linearities, thereby reducing the domain gap between the simplified testbed and the full MagLev simulator.

7.4 Discussion of Collaboration

For my specific focus on Reinforcement Learning, the collaboration was instrumental in establishing scientific rigor. The simultaneous development of high-performance MPC and Acados solvers by my group members provided a "gold standard" benchmark, allowing me to validate my RL agent's performance against real classical control data rather than theoretical estimates. Furthermore, the strict division of labor enabled me to focus exclusively on hyperparameter optimization and architectural search, ensuring the RL policy was competitive with these established baselines without needing to implement the entire control stack myself.

However, the project's execution highlighted the necessity of early-stage planning. The workflow was largely reactive, with integration points often defined dynamically as the semester progressed. In retrospect, a comprehensive long-term roadmap established at the outset would have significantly streamlined decision-making and ensured a smoother integration of our respective domains, avoiding the ad-hoc adjustments necessitated by our parallel development.

7.5 Recommendations for Further Work

Based on the challenges identified during this project, the following roadmap is recommended for the continued development of the RL component for the MagLev lab.

Short-term

- **Observation Stacking:** To potentially lower the 1000 Hz frequency requirement for the Pure RL agent, implementing observation stacking (providing the agent with a history of the last N states) should be investigated. This may allow the agent to infer higher-order dynamics (velocity/acceleration) more accurately from the available data.

Medium-term

- **Sim-to-Real Transfer (Pure RL Controller):** Although the Hybrid solution is easier to deploy, the ultimate objective remains deploying the Pure RL agent to the physical Maggy hardware. A critical verification step is profiling the *combined* execution time of the estimator and the RL agent. Given the hardware limit of 2000 Hz, the total computation time must strictly remain under 0.5 ms (or 1 ms for 1000 Hz). If the estimator consumes too much of this budget, optimization or alternative estimation strategies will be necessary.

- **Reward Function Engineering for Comfort:** The current reward functions prioritize survival and tracking accuracy. Future work should incorporate terms to minimize "jerk" (change in acceleration) and control effort variations to produce smoother, more natural motion profiles.

Long-term

- **Complex 6-DOF Trajectory Tracking:** Once stabilization is robust, the curriculum should expand to tracking complex 3D paths (e.g., figure-eights or spirals) involving simultaneous rotation and translation. This is where RL is expected to show significant advantages over linear control methods by learning to exploit non-linear magnetic coupling.

- **Integration into the "Take-home Maglev Lab":** The final goal is to package the trained agents and the training pipeline into a user-friendly module for the lab. This would allow students to train their own agents or compete against pre-trained baselines, fulfilling the educational objective of the project.

Chapter 8

Description of Use of Artificial Intelligence

In working on this assignment, I have used AI tools in the following ways: I used the Large Language Model (LLM) Gemini to assist with coding and text generation and Consensus.app to do literature research. AI was used to suggest phrasing and provide language improvements, but all content was reviewed, edited, and academically assessed by me. AI was not used to answer the assignment in its entirety. I am solely responsible for the academic content of the assignment.

Appendix A

Acronyms

AC Actor-Critic

AI Artificial Intelligence

DL Deep Learning

DOF Degrees of Freedom

EKF Extended Kalman Filter

LLM Large Language Model

LQR Linear-Quadratic Regulator

MagLev Magnetic Levitation

MDP Markov Decision Process

MHE Moving Horizon Estimation

MIMO Multiple-Input Multiple-Output

ML Machine Learning

MLP Multilayer Perceptron

MPC Model Predictive Control

NLP Nonlinear Program

NN Neural Network

OCP Optimal Control Problem

PG Policy Gradient

PID Proportional-Integral-Derivative

PPO Proximal Policy Optimization

ReLU Rectified Linear Unit

RL Reinforcement Learning

RMPC Robust Model Predictive Control

RMSE Root Mean Squared Error

Bibliography

[1] Hans Alvar Engmark and Kiet Tuan Hoang. Modeling and control of a magnetic levitation platform. In *Proceedings of the 22nd IFAC World Congress*, pages 7276–7281, Yokohama, Japan, 2023. IFAC.

[2] Hans Alvar Engmark. Take-home-Maglev-lab: A small lab for students in control engineering. `https://github.com/Hansolini/Take-home-Maglev-lab`, 2025. Accessed: 2025-12-11.

[3] Thomas Bächle, Sebastian Hentzelt, and Knut Graichen. Nonlinear model predictive control of a magnetic levitation system. *Control Engineering Practice*, 21(9):1250–1258, 2013.

[4] Bruno E. Silva and Ramiro S. Barbosa. Experiments with neural networks in the identification and control of a magnetic levitation system using a low-cost platform. *Applied Sciences*, 11(6), 2021.

[5] Ørnulf Damsgaard. Moving horizon state estimation for predictive control of magnetic levitation systems. Master's thesis, Norwegian University of Science and Technology (NTNU), June 2025.

[6] Mikael Mjelde. Robust model predictive control for a magnetic levitation system. Master's thesis, Norwegian University of Science and Technology (NTNU), June 2025.

[7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. `http://www.deeplearningbook.org`.

[8] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018. `http://incompleteideas.net/book/the-book-2nd.html`.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[10] Marcin Andrychowicz, Anton Raichuk, Piotr Staczyk, Manu Orsini, Sertan Girgin, Raphael Marinier, Léonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, Sylvain

Gelly, and Olivier Bachem. What matters in on-policy reinforcement learning? a large-scale empirical study, 2020.

[11] Fabio Pardo, Arash Tavakoli, Vitaly Levdik, and Petar Kormushev. Time limits in reinforcement learning, 2022.

[12] Benjamin Karg and Sergio Lucia. Efficient representation and approximation of model predictive control laws via deep learning. *IEEE Transactions on Cybernetics*, 50(9):38663878, September 2020.

[13] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):35213526, March 2017.

[14] Vijay Konda and John Tsitsiklis. Actor-critic algorithms. In S. Solla, T. Leen, and K. Müller, editors, *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999.